A Project Report On

CAN AI TAKE A JOKE - OR MAKE ONE? A STUDY OF HUMOR GENERATION

Submitted In partial fulfilment of the requirements for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

Submitted By

GUBBALA SYAM GANESH

Reg No: 246T1F0055

Under the esteemed guidance of

Mrs. M. SUJANA PRIYA DARSHINI M.TECH

Assistant Professor



DEPARTMENT OF MASTER OF COMPUTER APPLICATION

PYDAH COLLEGE OF ENGINEERING

(Approved by AICTE, New Delhi and Permanently Affiliated to JNTUK, Kakinada)
Accredited by NAAC "A" & An AUTONOMOUS Institute
PATAVALA, YANAM ROAD KAKINADA, 533461, Kakinada Dist.

2024-2026

PYDAH COLLEGE OF ENGINEERING

(Approved by AICTE, New Delhi and Permanently Affiliated to JNTUK, Kakinada)
Accredited by NAAC "A" & An AUTONOMOUS Institute
PATAVALA, YANAM ROAD KAKINADA, 533461, Kakinada Dist.
2024-2026

DEPARTMENT OF MASTER OF COMPUTER APPLICATION



BONAFIDE CERTIFICATE

This is to certify that the Project Entitled **"Can AI Take a Joke - Or Make One? A Study of Humor Generation"** is the Bonafide record of work done by **GUBBALA SYAM GANESH**, **ROLL NO: 246T1F0055** in partial fulfilment of the requirement for the award of the degree of **MASTER OF COMPUTER APPLICATIONS** in Pydah College of Engineering, Patavala, affiliated to Jawaharlal Nehru Technological University, KAKINADA.

It is Certified further that this work reported here in does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier date on this or any other candidate.

INTERNAL GUIDE **HEAD OF THE DEPARTMENT**

EXTERNAL EXAMINER

ACKNOWLEDGEMENTS

I would like to take this opportunity to express my gratitude for the successful completion of my project work entitled **"Can AI Take a Joke - Or Make One? A Study of Humor Generation."**

I am deeply thankful to our honorable Chairman of the institution **Sri P. SRI RAM Sir** for providing the necessary facilities and infrastructure to carry out this project successfully.

I am grateful to our Principal **Prof. Dr. P. V. SURYA PRAKASH** for his encouragement and support throughout the project.

I sincerely thank **Mr. P. CHAKRADHARARAO** Head of the Department, whose motivation and constant guidance helped me pursue this project in the field of artificial intelligence and natural language processing.

I am very much obliged and thankful to my internal guide Mrs. **M. SUJANA PRIYA DARSHINI** for providing this opportunity and for their valuable guidance and suggestions throughout the project. Their expertise and encouragement were invaluable.

My heartfelt thanks to my parents for their unwavering support and dedication. Because of their hard work, I have received opportunities beyond my wildest dreams.

Finally, I express my gratitude to all my professors, PRC members, and friends who supported me during the completion of this project

GUBBALA SYAM GANESH
(246T1F0055)

CONTENTS

S.NO.	CHAPTER	PAGE NO
	LIST OF FIGURES	6
	LIST OF SCREENS	7
	ABSTRACT	8
1.	**INTRODUCTION**	**9**
	1.1 Introduction	10
2.	**LITERATURE SURVEY**	**11**
	2.1Introduction	12
	2.2 Existing System	14
	2.3 Proposed System	15
3.	**SYSTEM ANALYSIS**	**16**
	3.1 Introduction	17
	3.2 Feasibility Study	18
	3.3 Input Design and Output Design	19
	3.4 Module Description	20
	3.5 System Architecture	21
4.	**REQUIREMENT SPECIFICATION**	**23**
	4.1 Hardware Requirements	24
	4.2 Software Requirements	25
	4.3 Technology Description	26
5	**SYSTEM DESIGN**	**27**
	5.1 Data Flow Diagrams	28
	5.2 UML Diagrams	29
	5.2.1 Use Case Diagram	30
	5.2.2 Class Diagram	31
	5.2.3 Sequence Diagram	32
	5.2.4 Activity Diagram	33

S.NO.	CHAPTER	PAGE NO
6	**IMPLEMENTATION AND RESULT**	**34**
	6.1 Introduction	35
	6.2 Explanation of Key Functions	37
	6.3 Code Implementation	39
7.	**TESTING AND VALIDATION**	**45**
	7.1 Introduction	46
	7.2 Types of Testing	47
8	**SAMPLE SCREENS**	**51**
9	**CONCLUSIONS**	
	9.1 Conclusion	61
	9.2 Future Scope	62
10	**BIBLIOGRAPHY**	**67**

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
Figure 3.1	SYSTEM ARCHITECTURE	**22**
Figure 5.1	SYSTEM ARCHITECTURE	**28**
Figure 5.2	DATA FLOW DIAGRAM	**29**
Figure 5.2	USE CASE DIAGRAM	**30**
Figure 5.3	CLASS DIAGRAM	**31**
Figure 5.4	SEQUENCE DIAGRAM	**32**
Figure 5.5	ACTIVITY DIAGRAM	**33**

LIST OF SCREENS

S.No	CONTENTS	PAGE NO
1	8.1 Home Page	52
2	8.2 User Registration Page	53
3	8.3 Admin Login Page	54
4	8.4 Admin Dashboard	55
5	8.5 Activate/Deactivate User Page	56
6	8.6 User Login Page	57
7	8.7 User Home Page	58
8	8.8 Joke Generation Page	59
9	8.9 Joke Output View	60

ABSTRACT

Humor generation has emerged as a fascinating domain within the broader scope of natural language generation and computational creativity. This project investigates whether artificial intelligence can not only recognize humor but also generate contextually appropriate jokes based on user-provided prompts. By integrating the **Gemini 2.5 Flash** model into a Django web application, the system allows users to input topics of interest and receive AI-generated jokes dynamically. The architecture combines user interaction, prompt engineering, and advanced language model capabilities to deliver humor in real time. The work addresses challenges in humor generation, including cultural sensitivity, contextual relevance, and the subjective nature of what is considered "funny." Compared to conventional joke datasets or template-based systems, the proposed approach offers more diversity, spontaneity, and personalization. Ultimately, the system demonstrates how generative AI can be applied beyond functional tasks—such as translation, summarization, and prediction—toward more creative and recreational applications, thereby making AI-driven experiences more engaging and enjoyable for end users.

Ultimately, the system demonstrates how generative AI can be applied beyond functional tasks—such as translation, summarization, and prediction—toward more creative and recreational applications, thereby making AI-driven experiences more engaging and enjoyable for end users across diverse demographic groups. By showcasing the ability of AI to generate humor, the project highlights the evolving role of artificial intelligence as a creative partner rather than merely a tool for automating routine or repetitive tasks, opening up new possibilities for human-AI collaboration in artistic and entertainment domains. The success of such applications depends not only on the underlying technical capabilities of the AI models but also on thoughtful interface design, user-centric interaction flows, and a deep understanding of the human factors that influence how people perceive and respond to AI-generated content. As generative AI continues to advance, systems like this humor generation platform will play an increasingly important role in bridging the gap between functional utility and emotional engagement, creating digital experiences that are not only useful but also genuinely enjoyable, surprising, and memorable for users around the world.

Chapter - 1
Introduction

INTRODUCTION

1.1 INTRODUCTION

Humor has long been considered a unique aspect of human intelligence, creativity, and social bonding. The ability to create, understand, and share jokes is not only a form of entertainment but also a critical component of communication and cultural expression. Throughout history, humor has served as a social lubricant, helping people connect, diffuse tension, and express complex ideas in accessible ways. The nuances of humor—timing, wordplay, cultural references, and shared context—make it one of the most challenging aspects of human communication to replicate computationally. With the rapid advancement of Artificial Intelligence (AI) and Natural Language Processing (NLP), researchers have begun to explore whether machines can also generate humor that resonates with humans. The emergence of large language models has opened new possibilities for computational creativity, enabling machines to produce text that is not only coherent and contextually appropriate but also creative and entertaining. This project, *"Can AI Take a Joke—Or Make One? A Study of Humor Generation"*, aims to design and implement a system where users can provide a topic and receive AI-generated jokes in return.

The project leverages state-of-the-art large language models such as Google's **Gemini 2.5 Flash** to explore the boundaries of machine creativity and humor comprehension. Unlike traditional rule-based joke generation, which often produced predictable or shallow outputs, modern generative AI models allow for contextual, topic-driven, and often surprisingly witty humor. The Gemini 2.5 Flash model, with its advanced natural language understanding and generation capabilities, can capture the subtleties of language that make humor effective—including wordplay, unexpected twists, and cultural references. This project not only demonstrates AI's capability to participate in a traditionally human-exclusive creative domain but also opens discussions on how humor can enhance human-AI interaction by making it more engaging, relatable, and enjoyable. The application is built using the Django web framework with Python, integrating the Gemini API for AI-powered joke generation. Users can register, log in, and submit topics to receive dynamically generated jokes. The system includes admin functionality for managing user accounts, ensuring a secure and interactive experience. The choice of Django as the web framework provides robust security features, session management, and database integration, making it ideal for building a user-centric web application.

Chapter - 2
Literature survey

LITERATURE SURVEY

2.1 INTRODUCTION

Computational humor has been an area of research for several decades, evolving from simple rule-based systems to sophisticated deep learning approaches. The field has attracted researchers from linguistics, computer science, psychology, and artificial intelligence, all seeking to understand what makes humor work and how it can be replicated computationally. This section reviews key contributions in the field of humor recognition and generation, tracing the evolution from early rule-based systems to modern deep learning approaches.

The study of computational humor dates back to the 1990s when researchers first began exploring whether computers could be programmed to understand and generate jokes. Early work focused on specific joke formats such as puns and riddles, using hand-crafted rules and templates to produce humorous outputs. While these systems demonstrated that computational humor was possible, they were limited by their rigidity and lack of creativity. As natural language processing techniques advanced, researchers began exploring machine learning approaches for humor detection and generation, leading to more flexible and capable systems.

The advent of large language models has revolutionized the field of computational humor. Models like GPT, BERT, and Gemini have demonstrated remarkable capabilities in understanding and generating human-like text, including humor. These models, trained on vast amounts of text data, have learned the patterns and structures that make humor effective, including wordplay, irony, sarcasm, and unexpected twists. This has opened new possibilities for creating AI systems that can generate original, contextually appropriate jokes in real time.

Mihalcea, R., & Strapparava, C. (2005) explored the early stages of computational humor, focusing on automatic recognition of humor in text. The authors used linguistic features, semantic incongruity, and lexical databases such as WordNet to identify humorous content. Their work was groundbreaking in demonstrating that humor recognition was feasible with a classification accuracy higher than random baselines. They identified key linguistic features associated with humor, including the presence of puns, the use of unexpected combinations, and the violation of semantic expectations. However, their work was limited to detection, not generation, highlighting the need for further research in creating humor computationally.

Binsted, K., & Ritchie, G. (1997) presented one of the earliest attempts at computational humor generation, a system for generating punning riddles based on phonetic and lexical similarities. The approach was entirely rule-based, relying on templates and wordplay mechanisms. Their system, called JAPE (Joke Analysis and Production Engine), was able to generate thousands of punning riddles by systematically exploring phonetic and orthographic similarities between words. While the generated riddles demonstrated a form of humor and were often grammatically correct, they lacked contextual adaptability and were repetitive. The system's outputs often felt mechanical and predictable, underlining the constraints of template-driven humor systems. Despite these limitations, JAPE established a foundation for computational humor generation and influenced subsequent research in the field.

He, H., & Glass, J. (2019) applied machine learning models to conversational data to identify humorous segments. Using linguistic cues, prosody, and contextual information, the authors trained models capable of detecting humor in dialogues. Their work was significant in moving beyond text-based humor detection to consider the role of conversation context and timing in humor recognition. They found that contextual information, such as the preceding conversation turns and the relationship between speakers, played a crucial role in determining whether an utterance was perceived as humorous. Their findings emphasized the role of context and timing in humor recognition, offering valuable insights for humor-aware dialogue systems. While effective at recognition, their system did not attempt humor generation, focusing instead on identifying humorous moments in natural conversation.

Winters, T., Nys, V., & De Schreye, D. (2018) proposed a general framework for humor generation using logical AI techniques. The authors integrated knowledge representation, linguistic resources, and rule-based structures to generate jokes across multiple formats. Their approach was notable for attempting to create a comprehensive framework that could handle different types of jokes, including puns, one-liners, and situational humor. They incorporated logical reasoning and world knowledge to generate jokes that were not only structurally correct but also semantically coherent. Although promising in creating structured humor, the system struggled with creativity, producing results that often felt mechanical and predictable. The study paved the way for combining symbolic and statistical AI in humor generation, recognizing that both rule-based and machine learning approaches have complementary strengths.

2.2 Existing System

Traditional humor generation systems primarily relied on rule-based methods or joke templates stored in databases. These systems often followed fixed structures such as "knock-knock jokes" or puns based on word substitutions. Some existing chatbots used pre-written joke libraries, randomly selecting and delivering jokes to users without true creativity or contextual understanding. While these approaches served as early experiments in computational humor, they were limited by their lack of adaptability, predictability, and inability to generate fresh humor based on user-provided topics.

The rule-based systems typically operated by identifying specific word patterns and substituting them according to predefined rules. For example, a pun generation system might identify words with similar pronunciations and replace them in a template to create a pun. While such systems could produce thousands of jokes, the outputs were often repetitive and lacked the spontaneity and creativity of human humor. The humor generated by these systems often felt forced and predictable because it followed rigid patterns that did not account for context, timing, or the subtleties of language.

Joke database systems, on the other hand, stored large collections of pre-written jokes and simply retrieved them based on keywords or categories. These systems were limited by the size of their joke database and could only deliver jokes that had been previously written and stored. They could not generate original jokes or adapt to specific topics or user preferences. The humor in these systems was essentially pre-packaged, lacking the freshness and relevance of jokes generated in response to a specific context.

The disadvantages of the existing system include heavy reliance on pre-stored jokes leading to repetition and lack of originality, inability to adapt humor to specific contexts, user preferences, or trending topics, production of humor that often feels mechanical, forced, or outdated, and limited interaction since users cannot meaningfully influence the joke generation process. The static nature of these systems also meant that they could not learn from user feedback or evolve over time to improve their humor. By leveraging the extensive training data of the Gemini model, the system can generate jokes that are appropriate for different contexts and cultures, although it still requires careful prompt engineering to ensure culturally sensitive outputs.

2.3 Proposed System

The proposed system leverages generative AI models like Google Gemini 2.5 Flash to create a dynamic humor generation platform. Users can provide topics and receive contextually relevant jokes generated in real time. The Django framework handles user interaction, session management, and display of AI-generated jokes, ensuring a seamless web-based experience. Unlike static joke databases, this system relies on deep learning-based NLP, allowing the model to capture nuances, wordplay, and cultural humor patterns. Additionally, the system ensures diversity in joke styles, producing puns, one-liners, and situational humor depending on the input.

The integration of the Gemini 2.5 Flash model represents a significant advancement over traditional humor generation approaches. Gemini 2.5 Flash is a state-of-the-art large language model that has been trained on a vast corpus of text, including many examples of humor. This training enables the model to understand the patterns and structures that make humor effective, including the use of unexpected twists, wordplay, irony, and cultural references. The model can generate original, contextually appropriate jokes that are not limited to pre-defined templates.

The system architecture consists of a Django web application that provides user authentication, session management, and a user-friendly interface. The core functionality is the AI joke generation module, which takes a user-provided topic, formulates a prompt, and sends it to the Gemini API. The generated joke is then displayed to the user in a clean, formatted card layout. The system also includes admin functionality for managing user accounts, allowing administrators to activate or deactivate users.

The advantages of the proposed system include generation of original and context-specific humor instead of repeating pre-written jokes, personalized engagement by letting users choose topics, utilization of state-of-the-art AI models for more natural and witty joke delivery, encouragement of human-AI interaction that feels entertaining and engaging, and the ability to adapt to modern language trends, slang, and cultural references. The system can also be extended with user feedback mechanisms to improve joke quality over time. The proposed system also addresses challenges in humor generation that earlier systems struggled with, including cultural sensitivity, contextual relevance, and the subjective nature of humor.

Chapter - 3
System Analysis

SYSTEM ANALYSIS

3.1 INTRODUCTION

System analysis is a critical phase in the software development life cycle that involves studying the existing system, identifying its limitations, and determining the requirements for the proposed solution. It helps in understanding user needs and designing a system that effectively addresses existing challenges. In the context of humor generation, traditional approaches often suffer from lack of originality, predictability, and inability to adapt to user preferences. The AI Humor Generation System is designed to overcome these limitations by providing a dynamic, AI-powered solution that generates contextually relevant jokes based on user input.

The system analysis phase involves understanding the problem domain, identifying the stakeholders, defining system requirements, and evaluating the feasibility of the proposed solution. For this project, the key stakeholders include end users who want to generate jokes on various topics, administrators who manage user accounts, and developers who maintain and enhance the system. The requirements were gathered through analysis of existing humor generation systems, identification of their limitations, and consideration of user needs for a more interactive and personalized humor generation experience.

The analysis phase also involves understanding the technical challenges involved in humor generation, including the need for natural language understanding, the ability to generate contextually appropriate responses, and the subjective nature of humor evaluation. The proposed system addresses these challenges by leveraging the Gemini 2.5 Flash model, which has been trained on vast amounts of text data and has demonstrated strong capabilities in natural language generation tasks. The AI Humor Generation System accepts various forms of input including user registration details such as name, loginid, password, mobile, email, locality, address, city, and state; user login credentials including loginid and password; admin login credentials; and topic input for joke generation. The input design includes validation for empty fields, format validation for email and mobile, and authentication validation for login. User-friendly error messages are displayed when validation fails. It is achieved by creating user-friendly screens for the data entry to handle large volume of data. The goal of designing input is to make data entry easier and to be free from errors. The data entry screen is designed in such a way that all the data manipulates can be performed. It also provides record viewing facilities. When the data is entered it will check for its validity.

3.2 FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential. The feasibility study examines three key aspects: economic feasibility, technical feasibility, and social feasibility.

Economic Feasibility: This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. The developed system is well within the budget and this was achieved because most of the technologies used are freely available. Python, Django, and SQLite are open-source and freely available, reducing development costs. The Gemini API usage costs are minimal for the scale of this project, and free tier options are available for development and testing. The hardware requirements are modest, requiring only a standard development machine and a web server for deployment.

Technical Feasibility: This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system. Python, Django, and the Gemini API are well-established technologies with extensive documentation and community support. The integration of these technologies is straightforward, with the Gemini API providing clear documentation and sample code for integration. The system architecture is modular, allowing for easy maintenance and future enhancements.

Social Feasibility: The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. The proposed application provides an intuitive and user-friendly interface that makes joke generation easy and entertaining. The system requires minimal training and is accessible through web browsers. The entertaining nature of the application is likely to increase user acceptance and engagement.

3.3 INPUT DESIGN AND OUTPUT DESIGN

Input Design: The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those steps are necessary to put transaction data into a usable form for processing can be achieved by inspecting the computer to read data from a written or printed document or it can occur by having people keying the data directly into the system. The design of input focuses on controlling the amount of input required, controlling the errors, avoiding delay, avoiding extra steps and keeping the process simple. The input is designed in such a way so that it provides security and ease of use with retaining the privacy. Input Design considered the following things: what data should be given as input, how the data should be arranged or coded, the dialog to guide the operating personnel in providing input, and methods for preparing input validations and steps to follow when error occur.

Output Design: A quality output is one which meets the requirements of the end user and presents the information clearly. In any system results of processing are communicated to the users and to other system through outputs. In output design it is determined how the information is to be displaced for immediate need and also the hard copy output. It is the most important and direct source information to the user. Efficient and intelligent output design improves the system's relationship to help user decision-making.

Designing computer output should proceed in an organized, well thought out manner; the right output must be developed while ensuring that each output element is designed so that people will find the system can use easily and effectively. When analysis design computer output, they should Identify the specific output that is needed to meet the requirements, select methods for presenting information, and create document, report, or other formats that contain information produced by the system. The output form of an information system should accomplish one or more of the following objectives: convey information about past activities, current status or projections of the future, signal important events, opportunities, problems, or warnings, trigger an action, and confirm an action.

The system generates various outputs including registration confirmation messages, login success or error messages, the user dashboard with options for joke generation, and the AI-generated joke displayed in a formatted card layout. The output design ensures that information is presented clearly and effectively, with appropriate visual styling and formatting to enhance user experience. The joke output is presented in a clean card layout with the topic displayed prominently above the generated joke.

3.4 MODULE DESCRIPTION

The AI Humor Generation System is divided into several functional modules that work together to provide comprehensive humor generation capabilities.

User Module: The User Module serves as the primary interface for users interacting with the humor generation system. Users can register and log in securely to access features such as generating jokes based on topics. The system allows users to input topics and receive AI-generated jokes dynamically. This interactive and entertaining interface aims to make AI-driven experiences more engaging and enjoyable. The user module handles all aspects of user interaction including registration, authentication, session management, and joke generation requests.

Admin Module: The Admin Module plays a crucial role in maintaining and managing the overall functionality of the platform. Administrators can monitor user activity, approve or manage user registrations, and oversee uploaded content for quality and relevance. Additionally, the admin is responsible for managing user accounts by activating or deactivating them. This ensures that only authorized and verified users can access the system's features. The admin module provides a dashboard for viewing all registered users and their status, with options to change user status from "waiting" to "activated" or from "activated" to "deactivated."

AI Joke Generation Module: The AI Joke Generation Module is the core component of the system that enables dynamic humor creation using generative AI. It accepts a topic from the user as input and formulates a prompt, which is then processed by the Google Gemini 2.5 Flash model to generate a contextually relevant and witty joke. This module ensures originality, adaptability, and entertainment by producing jokes in real time instead of relying on predefined templates. It also handles cases where no topic is provided, defaulting to general humor, thereby ensuring a continuous and engaging user experience. The module includes prompt engineering to optimize joke quality, error handling for API failures, and formatting of the generated joke for display.

Database Module: The Database Module handles all database operations including user registration, authentication, and account management. It uses SQLite as the database engine and provides methods for inserting user data, validating login credentials, retrieving user status, and updating user status. The module ensures data integrity and security through parameterized queries and input validation.

3.5 SYSTEM ARCHITECTURE

The architecture of the AI Humor Generation System follows the Django MVC (Model-View-Controller) design pattern with integration of the Gemini API. The architecture is designed for maintainability, scalability, and reliability.

Presentation Layer (View): The presentation layer contains HTML templates that provide the user interface. The templates include base.html for the common layout and navigation, index.html for the home page, UserLogin.html for user authentication, UserRegistrations.html for user registration, AdminLogin.html for admin authentication, AdminHome.html for the admin dashboard, and users/UserHomePage.html and users/predict.html for user dashboard and joke generation. The templates use Bootstrap CSS for responsive design and custom CSS for background styling and visual enhancements.

Application Layer (Controller): The application layer contains Django views that handle HTTP requests and manage application flow. The views include UserRegisterActions for processing registration requests, UserLoginCheck for authenticating users, AdminLoginCheck for authenticating administrators, and makejoke for generating jokes. These views handle request validation, business logic execution, and response rendering.

Business Layer (Model): The business layer contains Django models that represent the data structures and business logic. The UserRegistrationModel represents user data with fields for id, name, loginid, password, mobile, email, locality, address, city, state, and status. The model provides methods for data validation and database operations.

AI Integration Layer: The AI integration layer provides the interface to the Google Gemini 2.5 Flash API. It handles API configuration, prompt formulation, request construction, response processing, and error handling. The layer encapsulates the complexity of API interaction, providing a clean interface for the application layer to use for joke generation.

Data Layer: The data layer uses SQLite as the database engine. It stores user registration information, authentication credentials, and account status. The data layer is accessed through Django's ORM, which provides a high-level abstraction for database operations.The architecture ensures separation of concerns, maintainability, and scalability. The modular design allows for easy modification and extension of functionality. The integration of the Gemini API demonstrates how cloud-based AI services can be incorporated into web applications to provide intelligent, real-time functionality.

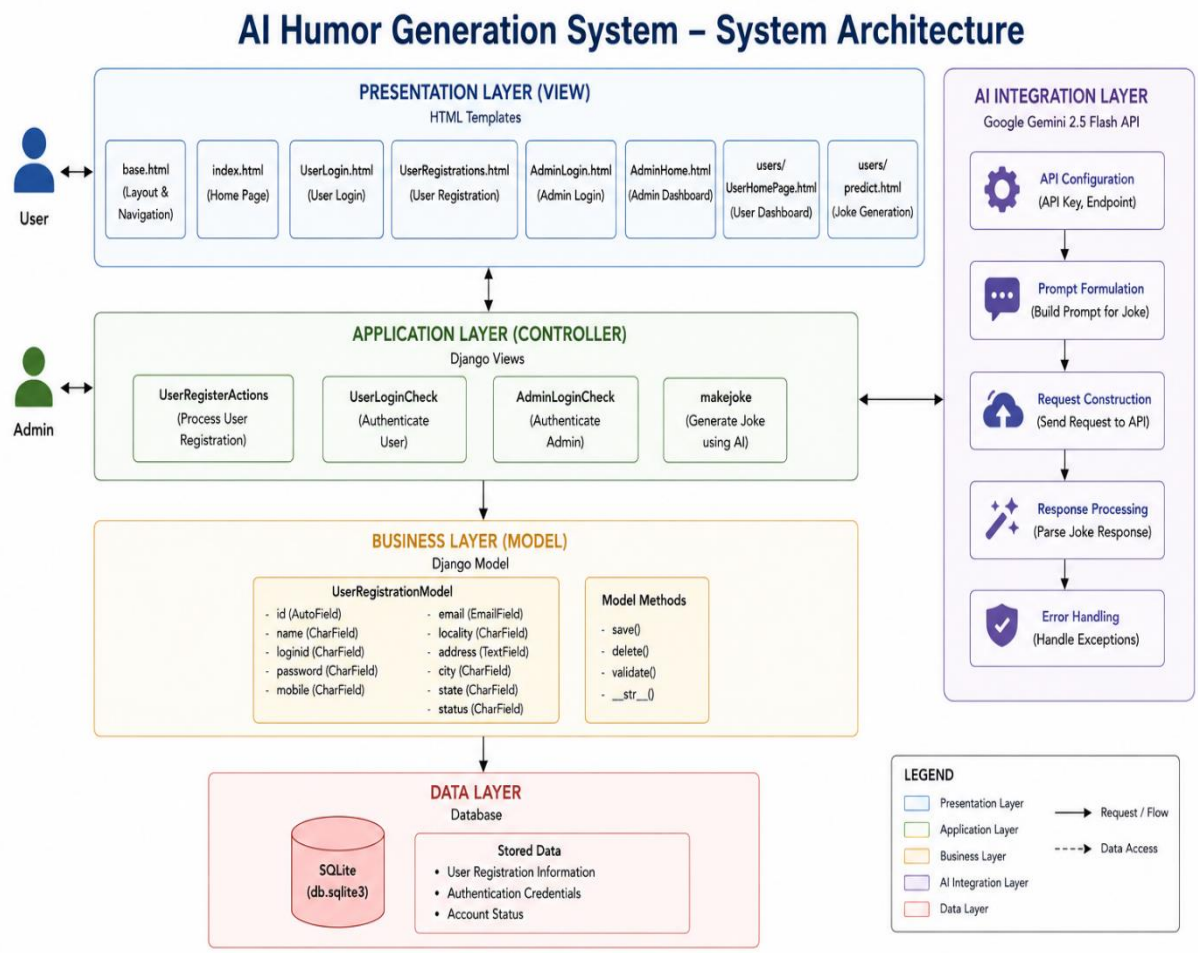


Figure 3.1: SYSTEM ARCHITECTURE

Chapter - 4

Requirement Specification

REQUIREMENT SPECIFICATION

4.1 HARDWARE REQUIREMENTS

Hardware requirements define the physical resources necessary for the development, deployment, and execution of the Cloud-Based File Storage System. Since the application is developed using Java Spring Boot and utilises AWS S3 cloud storage, the hardware requirements are moderate and can be satisfied using standard computing systems. A computer with an Intel Core i3 or later processor is sufficient for development and testing activities.

A minimum of 4 GB of RAM is required; 8 GB is recommended for smooth performance when running the Spring Boot application, the database server, and development tools simultaneously. The system requires at least 100 GB of available storage space to install software, store project files, and maintain local development resources. A stable internet connection is essential because the application communicates with AWS cloud services for file storage and retrieval operations.

The hardware configuration used for the development of this project consists of a personal computer equipped with an Intel processor, 8 GB of RAM, a 256 GB SSD, and broadband internet connectivity. These resources provide sufficient computing power to effectively develop, test, and demonstrate the cloud-based file storage application. System is developed using Python programming language with Django web framework

COMPONENT	SPECIFICATION
Processor	Intel Core i3/i5/i7
RAM	4 GB Minimum, 8 GB Recommended
Storage	100 GB HDD/SSD
Internet	Broadband Connection
Monitor	15" or Above

4.2 SOFTWARE REQUIREMENTS

Software requirements represent the collection of operating systems, development tools, programming languages, frameworks, and libraries necessary for implementing the project. The AI Humor Generation. The application uses SQLite as the database, HTML, CSS, and JavaScript for the front-end, and Google Gemini API for AI-powered joke generation. Python is used as the primary programming language due to its readability, extensive library support, and strong community. Django is chosen as the web framework for its robust security features, built-in admin interface, and comprehensive ORM. SQLite is used as the database for its simplicity and zero-configuration requirement. The front-end technologies provide a responsive and visually appealing user interface.

SOFTWARE	SPECIFICATION
Operating System	Windows 10 / Linux / macOS
Coding Language	Python 3.8+
Web Framework	Django 4.0+
Front-End	HTML, CSS, JavaScript
AI Integration	Google Gemini API (gemini-2.5-flash)
Database	SQLite
IDE	Visual Studio Code / PyCharm
Browser	Chrome / Firefox / Edge / Safari
Version Control	Git

4.3 TECHNOLOGY DESCRIPTION

The AI Humor Generation System is developed using modern technologies that ensure performance, maintainability, and reliability.

Python: Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages. Python is processed at runtime by the interpreter, eliminating the need for compilation. It supports interactive mode for direct execution of commands. Python supports object-oriented style or technique of programming that encapsulates code within objects. It is a great language for beginner-level programmers and supports the development of a wide range of applications from simple text processing to web applications to games. Python features include easy-to-learn syntax, easy-to-read code, easy-to-maintain source code, a broad standard library, interactive mode for testing and debugging, portability across hardware platforms, extendability through low-level modules, database interfaces, GUI programming support, and scalability for large programs.

Django: Django is a high-level Python Web framework that encourages rapid development and clean, pragmatic design. Built by experienced developers, it takes care of much of the hassle of Web development, so you can focus on writing your app without needing to reinvent the wheel. It's free and open source. Django's primary goal is to ease the creation of complex, database-driven websites. Django emphasizes reusability and "pluggability" of components, rapid development, and the principle of don't repeat yourself. Python is used throughout, even for settings files and data models. Django also provides an optional administrative create, read, update and delete interface that is generated dynamically through introspection and configured via admin models. Django includes a powerful ORM for database interactions, a template system for rendering HTML, URL routing for clean URLs, form handling for input validation, and authentication and authorization for security.

HTML, CSS, JavaScript: HTML provides the structure and semantics of the web pages. CSS provides styling and visual design including background images, gradients, animations, and responsive layouts. JavaScript provides interactivity and dynamic functionality including form validation and AJAX requests.

Chapter - 5
System Design

SYSTEM DESIGN

5.1 DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) is a graphical representation that illustrates how data moves through a system. It helps in understanding the flow of information between users, processes, and data stores. The AI Humor Generation System uses a structured data flow mechanism to manage user interactions and data operations.

The DFD is also called as bubble chart. It is a simple graphical formalism that can be used to represent a system in terms of input data to the system, various processing carried out on this data, and the output data generated by this system. The data flow diagram is one of the most important modeling tools used to model the system components including system processes, data used by the process, external entities that interact with the system, and information flows in the system. DFD shows how the information moves through the system and how it is modified by a series of transformations. It is a graphical technique that depicts information flow and the transformations that are applied as data moves from input to output. DFD may be used to represent a system at any level of abstraction and may be partitioned into levels that represent increasing information flo and functional detail.

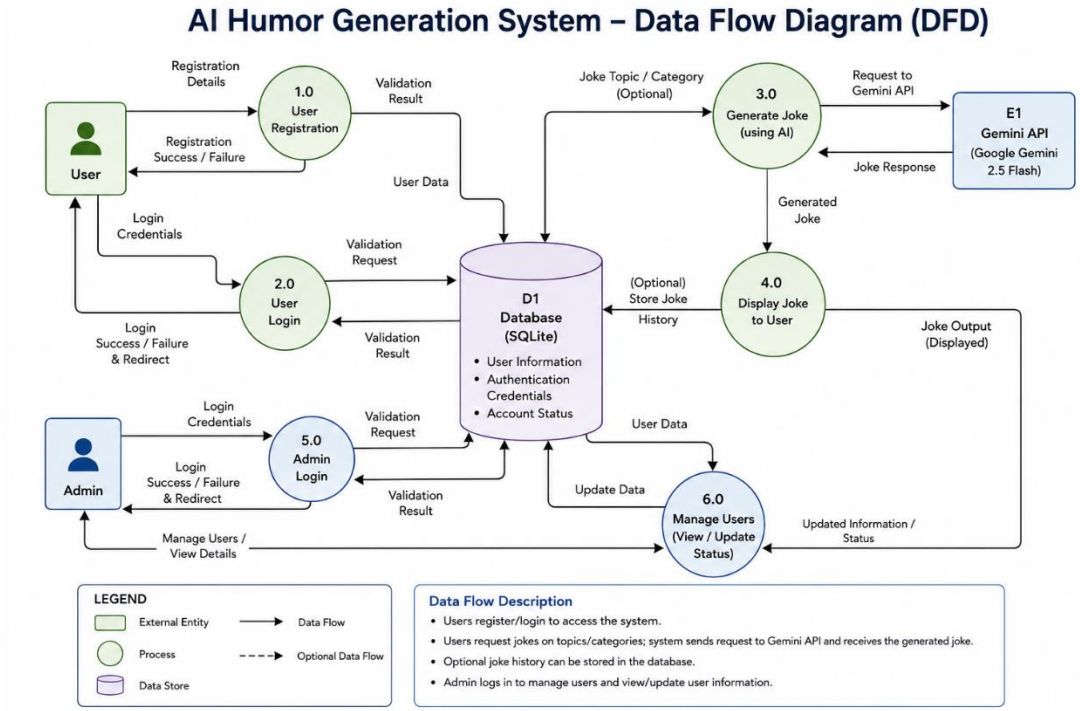


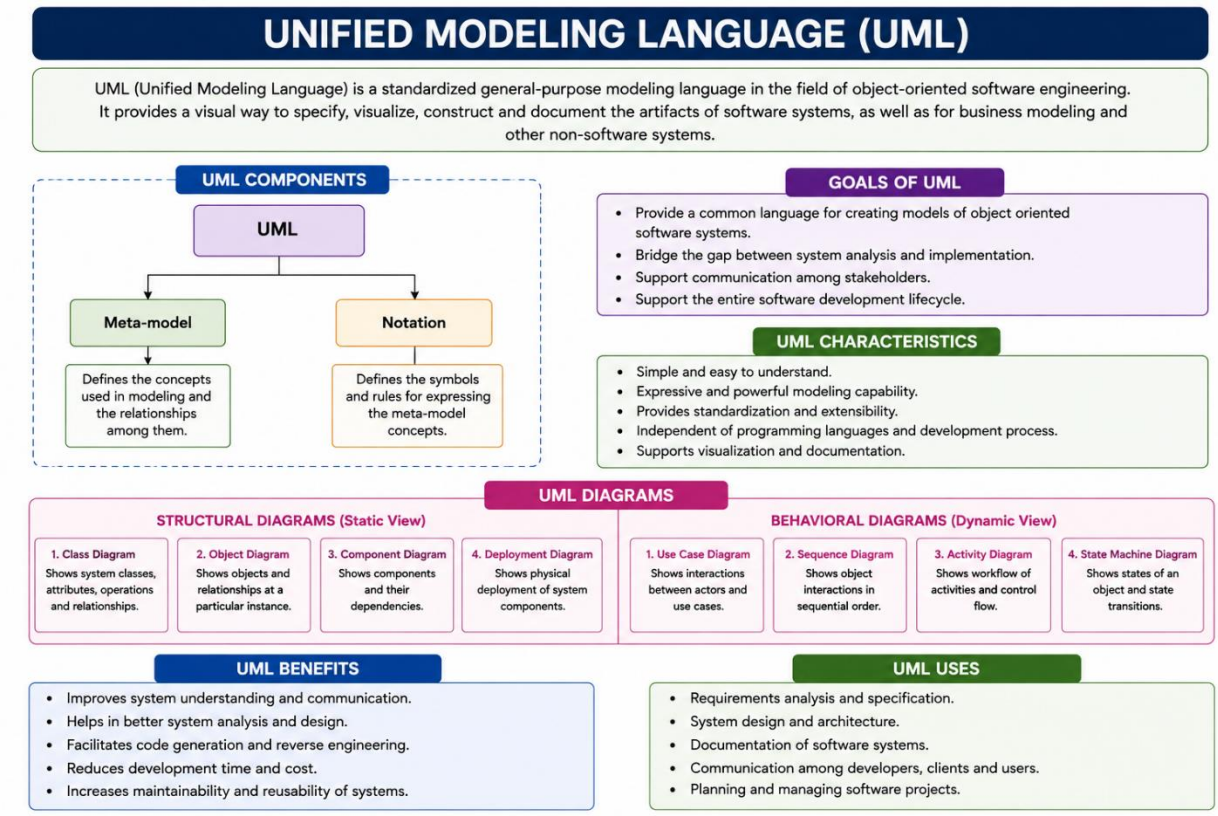
Figure 5.1: SYSTEM ARCHITECTURE

5.2 UML DIAGRAMS

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group. The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation.

The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems. The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems. The UML is a very important part of developing objects oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

.



.

Figure 5.2: DATA FLOW DIAGRAM

5.2.1 USE CASE DIAGRAM

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

The Use Case Diagram represents the interactions between actors and the AI Humor Generation System. The primary actors in the system are the Admin and the User. The Admin can perform operations such as login to the system, view users, and activate or deactivate user accounts. The User can perform operations such as register as a new user, login to the system, and generate jokes based on topics. Each use case fulfills a specific application requirement and contributes to the overall functionality of the humor generation system. The admin login use case provides secure access for administrators. The view users and activate/deactivate users use cases allow administrators to manage user accounts.

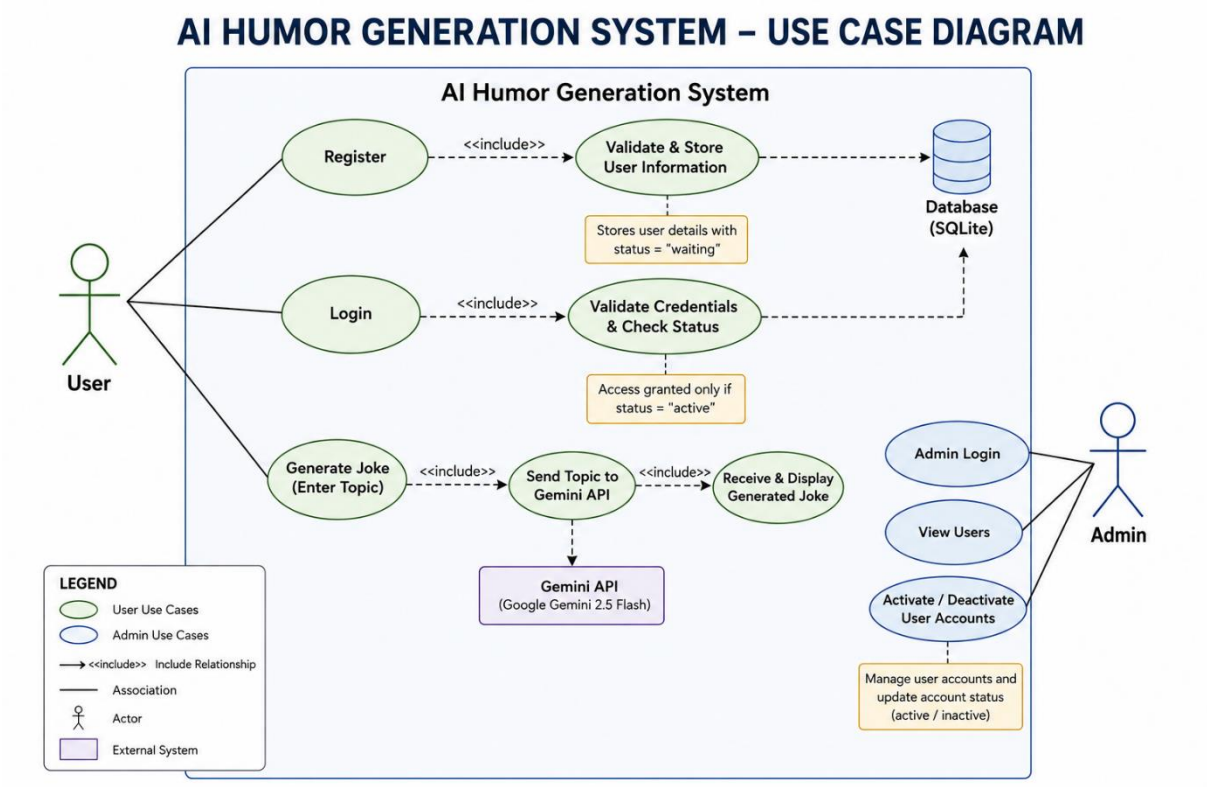


Figure 5.3: USE CASE DIAGRAM

5.2.2 CLASS DIAGRAM

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

The Class Diagram represents the static structure of the AI Humor Generation System. The primary classes include UserRegistrationModel with attributes id, name, loginid, password, mobile, email, locality, address, city, state, and status. The class provides methods for object creation and data access. The views include User Register Actions for handling registration requests, User Login Check for authenticating users and checking account status, Admin Login Check for authenticating administrators, and make joke for generating jokes using the Gemini API. The relationships among these classes ensure modularity, reusability, and maintainability of the application. The User Registration Model class represents the user entity with all the attributes required for user management. The status attribute indicates whether the user account is "waiting", "activated", or "deactivated". The views interact with the model through Django's ORM to perform CRUD operations. The makejoke view has a dependency on the Gemini API for joke generation, encapsulating the API interaction logic.

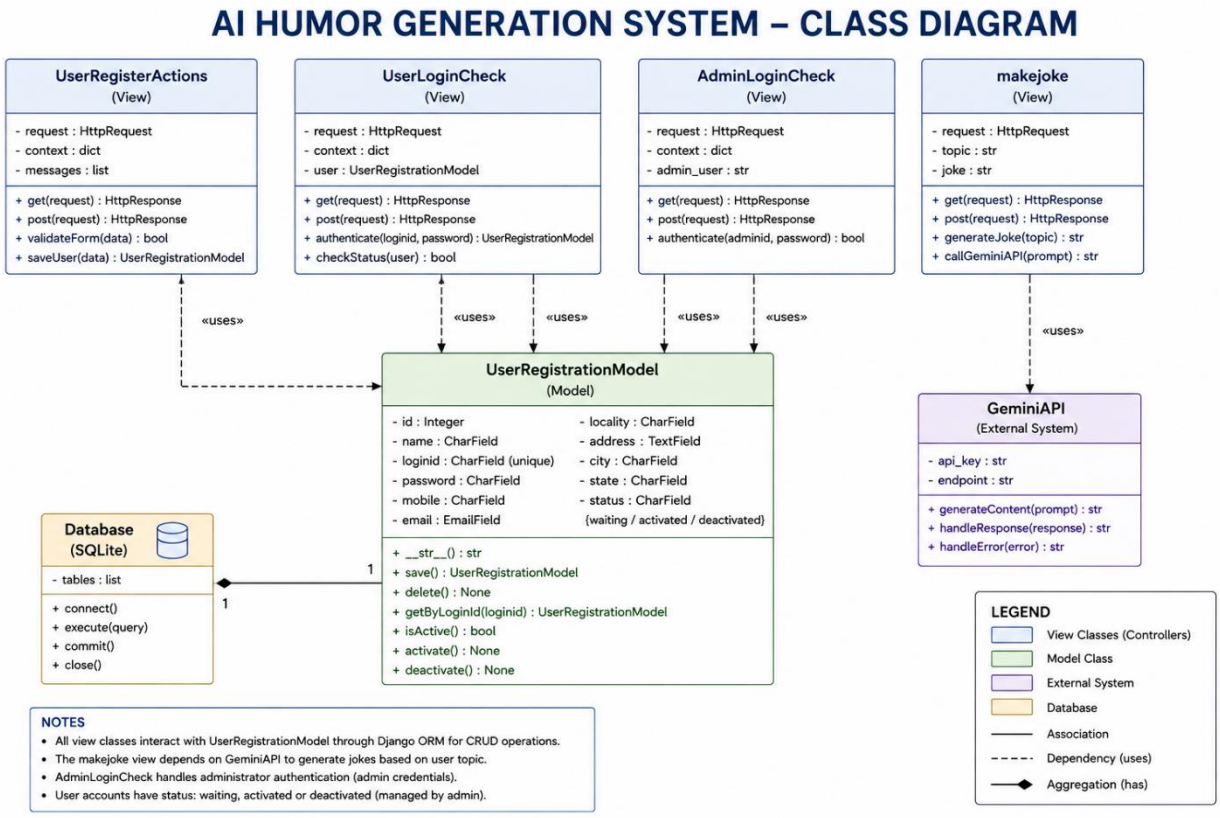


Figure 5.4: CLASS DIAGRAM

5.2.3 SEQUENCE DIAGRAM

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

The Sequence Diagram illustrates the interaction between system components over time. In the login process, the user enters credentials in the login page and submits the form. The UserLoginCheck view receives the request and queries the UserRegistrationModel to validate the credentials. If the credentials are valid and the account status is "activated", the view creates a session, stores user information, and redirects to the user home page. If the credentials are invalid, an error message is displayed. If the account is not activated, a message is displayed indicating that the account is pending activation.

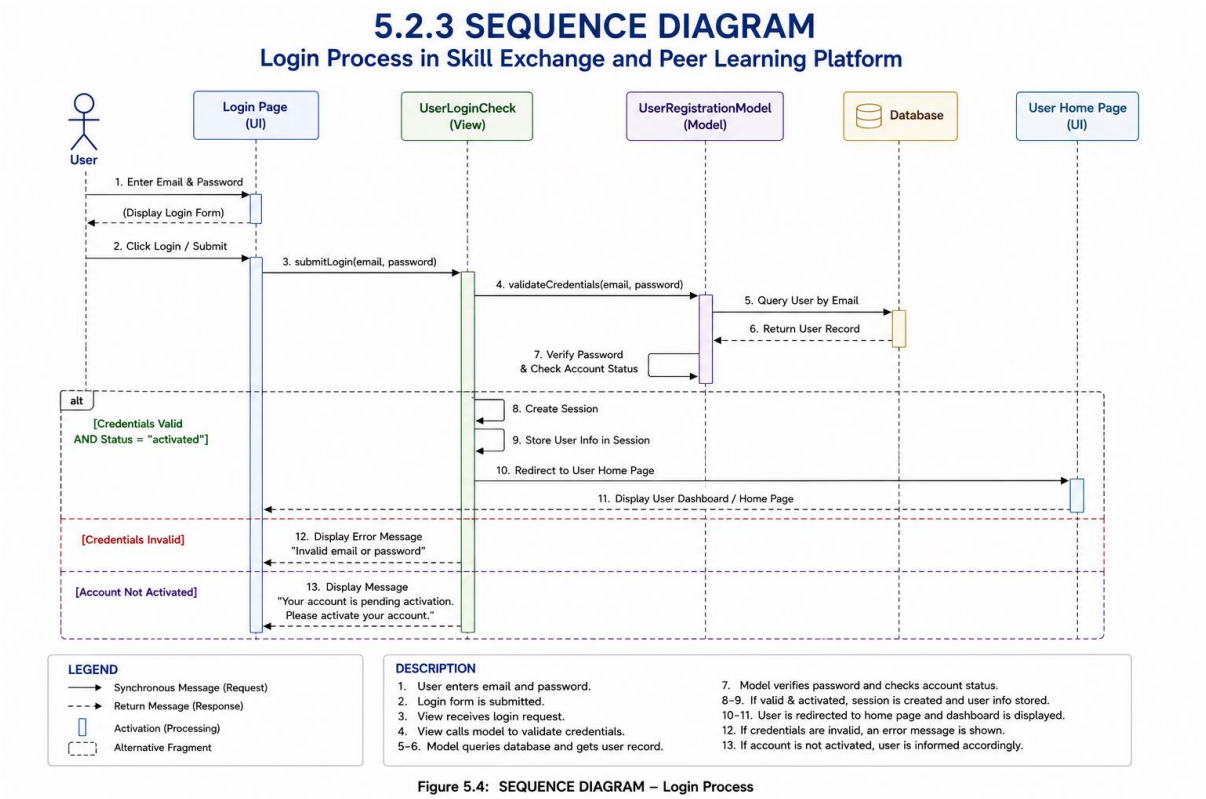


Figure 5.4: SEQUENCE DIAGRAM – Login Process

Figure 5.5: SEQUENCE DIAGRAM

5.2.4 ACTIVITY DIAGRAM

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

The Activity Diagram for the AI Humor Generation System shows the complete workflow of the application. The workflow begins with the user accessing the home page. The user can choose to login, register, or access the admin login. If registering, the user fills the registration form and submits. The system validates the input and saves the user to the database with status "waiting". The user then proceeds to login with valid credentials. The system checks the user status; if "activated", the user accesses the user dashboard. The user can generate jokes by entering a topic. The system processes the request through the Gemini API and displays the generated joke.

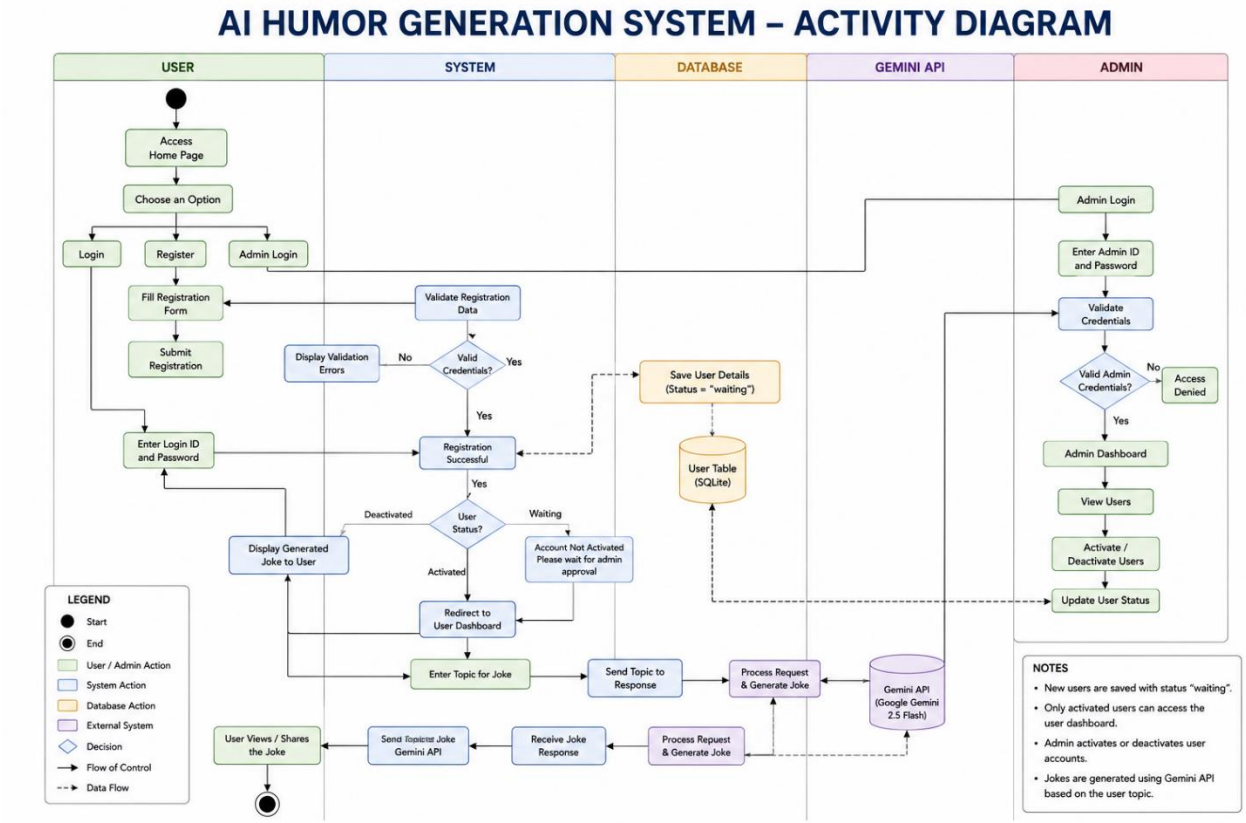


Figure 5.6: ACTIVITY DIAGRAM

Chapter - 6

Implementations and Results

IMPLEMENTATION AND RESULT

6.1 INTRODUCTION

The implementation phase is one of the most important stages in software development, where the designed system is transformed into a fully functional application, translating the architectural blueprints and design specifications into working code that delivers tangible value to end users through seamless interactions and reliable performance. The AI Humor Generation System was implemented using Python with Django framework, SQLite database, and Google Gemini API integration, leveraging these powerful and well-established technologies to create a robust, scalable, and maintainable web application that provides a fast and responsive user experience with an engaging and entertaining interface. The primary objective of the implementation was to provide a robust, user-friendly, and entertaining AI-powered joke generation solution, ensuring that users can input topics of interest and receive contextually appropriate, humorous responses generated by state-of-the-art artificial intelligence while maintaining a seamless and enjoyable interaction experience. The implementation also focused on creating a modern, visually appealing interface that communicates playfulness and creativity while ensuring that all interactive elements are intuitive and responsive, encouraging users to explore different joke topics and engage repeatedly with the platform.

The application follows a modular architecture consisting of Django views, templates, and models, adhering to best practices in software design that promote code organization, reusability, and maintainability throughout the development lifecycle. The HTML templates provide the structure with CSS styling and JavaScript for interactivity, creating a clean and engaging user interface that supports intuitive navigation and responsive interaction with the joke generation features. The Django views handle business logic, request processing, and database operations, serving as the central coordination point that directs user interactions to the appropriate handlers and returns the appropriate responses, ensuring that all user requests are properly processed and that the joke generation workflow executes correctly. The Gemini API integration provides AI-powered joke generation capabilities, connecting the application to Google's advanced language models that deliver intelligent, context-aware humor generation that adapts to user-provided topics and preferences. The implementation was carried out incrementally, starting with the core authentication and registration functionality, followed by the Gemini API integration, and finally the admin management features, ensuring that each component was properly tested and validated before integration with the complete system.

The development environment used for implementation included Visual Studio Code as the IDE, providing a lightweight yet powerful editor with extensive extension support for Python and Django development that streamlined the coding process and improved developer productivity. Python 3.10 served as the programming language, offering modern language features and extensive library support that enabled efficient development of the web application and seamless integration with the Gemini API. Django 4.2 provided the web framework, offering robust features for URL routing, template rendering, and database management that accelerated development and ensured reliable application behavior.

SQLite served as the database, providing lightweight, file-based data storage that simplified deployment and testing without requiring complex database configuration. The Gemini API was accessed using the google-generativeai Python library, enabling straightforward integration with Google's AI services and supporting efficient generation of humorous content based on user prompts. The application was tested locally using Django's development server before being deployed for production use, ensuring that all features were thoroughly validated and that the system performed reliably under various operating conditions.

Activate/Deactivate User Function: This function allows administrators to manage user accounts, providing comprehensive control over user access and ensuring that only verified users can interact with the joke generation system. The admin can view all registered users and change their status from "waiting" to "activated" or from "activated" to "deactivated," enabling flexible user management that supports moderation and access control for the platform.

This provides control over which users can access the system, ensuring that administrators can approve new registrations, suspend problematic users, or deactivate accounts as needed to maintain a safe and enjoyable user environment. The function updates the status field in the UserRegistrationModel and saves the changes to the database, ensuring that status changes are immediately reflected in the system and that user access permissions are updated consistently. A success message is displayed confirming the status change, providing clear feedback to administrators that their action has been completed successfully and that the user's account status has been updated accordingly.

6.2 EXPLANATION OF KEY FUNCTIONS

UserRegisterActions Function: This function handles user registration, providing the entry point for new users to create accounts and gain access to the joke generation system through a streamlined and secure registration process. It receives user details from the registration form including name, loginid, password, mobile, email, locality, address, city, and state, collecting all necessary information to establish a complete user profile with appropriate access permissions. It creates a new UserRegistrationModel object with status set to "waiting" and saves it to the database, ensuring that new registrations are properly recorded and that user accounts are created with a pending status that requires administrative approval before access is granted. Upon successful registration, a success message is displayed to the user, providing clear confirmation that their account has been created and informing them that they will need to wait for admin activation before they can access the system. The function includes error handling for duplicate loginid and validation for required fields, preventing duplicate account creation and ensuring that all necessary information is provided for complete and valid user profiles. The status is initially set to "waiting", requiring admin activation before the user can access the system, providing a moderation mechanism to ensure only verified users can use the system and preventing unauthorized access by unapproved accounts.

UserLoginCheck Function: This function handles user authentication, providing secure access to the joke generation system and ensuring that only authorized users with approved accounts can interact with the AI humor generation features. It receives loginid and password from the login form, queries the database to validate the credentials, and checks the user status, verifying both the correctness of the credentials and the activation status of the user account. If the credentials are valid and the status is "activated", the user's session is created with user details including id, name, loginid, and email, establishing a secure session that maintains user state across requests and provides access to personalized features. The user is then redirected to the user home page, where they can immediately begin generating jokes and exploring the system's capabilities. If the status is "waiting" or "deactivated", a message is displayed indicating that the account is not activated, informing the user that their registration is pending approval or that their account has been suspended. If the credentials are invalid, an error message is displayed, providing clear feedback that helps users correct their input and successfully authenticate. The function uses Django's session framework to maintain user state across requests, ensuring a seamless and secure user experience that does not require repeated authentication for each interaction.

AdminLoginCheck Function: This function handles admin authentication, providing secure access to administrative features that enable moderation and management of the user base for the joke generation platform. It receives loginid and password from the admin login form and validates them against predefined admin credentials stored in the application logic, ensuring that only authorized administrators can access the admin dashboard and perform user management operations. Upon successful authentication, the admin is redirected to the admin dashboard, where they can view all registered users, manage account statuses, and oversee system operations. If the credentials are invalid, an error message is displayed, providing clear feedback that helps administrators correct their input and successfully authenticate. The admin credentials are hardcoded for simplicity in this implementation, but in a production system, they would be stored securely in the database with proper password hashing and additional security measures such as multi-factor authentication to ensure robust protection of administrative access.

makejoke Function: This function is the core of the AI joke generation module, providing the intelligent humor generation capabilities that deliver personalized, contextually appropriate jokes based on user-provided topics and preferences. It receives a topic from the user via POST request, capturing the user's interest and enabling personalized joke generation that adapts to the specific topic they want to explore. It formulates a prompt based on the topic: if a topic is provided, the prompt is "Tell me a funny joke about [topic]"; if no topic is provided, the prompt is "Tell me a funny joke," ensuring that the AI receives clear, specific instructions that guide the generation of appropriate and relevant humorous content. The function configures the Gemini API with the API key, securely accessing Google's advanced language models through proper authentication and authorization mechanisms. It creates a GenerativeModel instance with the "gemini-2.5-flash" model, leveraging a fast, efficient model specifically designed for quick, high-quality text generation that delivers jokes with minimal latency. The function generates content using the prompt and extracts the generated joke from the response, parsing the AI output to provide clean, well-formatted joke content that is ready for display to the user. The joke is then passed to the template for display, presenting the generated humor in an engaging and visually appealing format that encourages user engagement. The function includes error handling for API failures, displaying a friendly error message if joke generation fails, ensuring that users receive clear feedback and are not left confused by technical issues, while also logging errors for debugging and system improvement purposes.

6.3 CODE IMPLEMENTATION

Base.html:

html

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8" />
   <meta name="viewport" content="width=device-width, initial-scale=1" />
   <title>ai humor joke generation</title>
   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="s
tylesheet" />
   <style>
      body {
         margin: 0;
         min-height: 100vh;
         font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
         color: #1e3a8a;
         background: url("{% static 'bg4.jpg' %}") no-repeat center center fixed;
         background-size: cover;
      }
      .navbar {
         background-color: rgba(30, 58, 138, 0.85);
         padding: 15px 20px;
         box-shadow: 0 4px 8px rgba(0, 0, 0, 0.25);
      }
      .navbar-brand {
         font-size: 1.8rem;
         font-weight: 700;
         color: #ffffff !important;
         text-transform: uppercase;
         letter-spacing: 1.5px;
      }
      .nav-link {
         color: #dbeafe !important;
         font-weight: 600;
         padding: 10px 18px;
         border-radius: 6px;
         transition: all 0.3s ease;
      }
```

```
.nav-link:hover {
    background-color: #60a5fa;
    color: #ffffff !important;
    transform: scale(1.05);
}
.content {
    padding: 40px 20px;
    margin-top: 20px;
    min-height: calc(100vh - 100px);
    display: flex;
    justify-content: center;
    align-items: flex-start;
}
.main-content {
    border-radius: 20px;
    padding: 40px 35px;
    max-width: 900px;
    width: 100%;
    color: #1e3a8a;
}
.btn-primary {
    background: linear-gradient(135deg, #4ade80, #16a34a);
    color: #fff;
    border: none;
    border-radius: 8px;
    padding: 12px 24px;
    font-weight: 600;
    transition: all 0.3s ease;
    box-shadow: 0 6px 15px rgba(0, 0, 0, 0.2);
}
.btn-primary:hover {
    background: linear-gradient(135deg, #22c55e, #15803d);
    transform: scale(1.05);
    color: #fff;
}
footer {
    text-align: center;
    padding: 20px 10px;
    color: #dbeafe;
    background-color: rgba(30, 58, 138, 0.9);
    width: 100%;
    font-weight: 500;
```

```
        }
    </style>
</head>
<body>
    <nav class="navbar navbar-expand-lg">
        <div class="container-fluid">
            <a class="navbar-brand" href="{% url 'index' %}">Joke Generation</a>
            <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-targe
t="#navbarNav">
                <span class="navbar-toggler-icon" style="filter: invert(1);"></span>
            </button>
            <div class="collapse navbar-collapse" id="navbarNav">
                <ul class="navbar-nav ms-auto">
                    <li class="nav-item"><a class="nav-link" href="{% url 'index' %}"> Home</a>
</li>
                    <li class="nav-item"><a class="nav-link" href="{% url 'UserLogin' %}"> User
Login</a></li>
                    <li class="nav-item"><a class="nav-link" href="{% url 'AdminLogin' %}"> Ad
min Login</a></li>
                    <li class="nav-item"><a class="nav-link" href="{% url 'UserRegisterForm' %}"
> Create Account</a></li>
                </ul>
            </div>
        </div>
    </nav>
    <div class="content">
        <div class="main-content">
            {% block contents %}
            {% endblock %}
        </div>
    </div>
    <footer>
        <p>&copy; ai_humor_joke_generation 2025. All rights reserved.</p>
    </footer>
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js">
</script>
</body>
</html>
```

views.py:

python

```
from .forms import PredictionForm
```

```
import os
from django.shortcuts import render
from django.conf import settings
from django.shortcuts import render, redirect
from .models import UserRegistrationModel
from django.contrib import messages

def UserRegisterActions(request):
    if request.method == 'POST':
        user = UserRegistrationModel(
            name=request.POST['name'],
            loginid=request.POST['loginid'],
            password=request.POST['password'],
            mobile=request.POST['mobile'],
            email=request.POST['email'],
            locality=request.POST['locality'],
            address=request.POST['address'],
            city=request.POST['city'],
            state=request.POST['state'],
            status='waiting'
        )
        user.save()
        messages.success(request, "Registration successful!")
    return render(request, 'UserRegistrations.html')

def UserLoginCheck(request):
    if request.method == "POST":
        loginid = request.POST.get('loginid')
        pswd = request.POST.get('pswd')
        try:
            check = UserRegistrationModel.objects.get(loginid=loginid, password=pswd)
            status = check.status
            if status == "activated":
                request.session['id'] = check.id
                request.session['loggeduser'] = check.name
                request.session['loginid'] = loginid
                request.session['email'] = check.email
                return render(request, 'users/UserHomePage.html', {})
            else:
                messages.success(request, 'Your Account Not activated')
                return render(request, 'UserLogin.html')
        except Exception as e:
```

```
            pass
        messages.success(request, 'Invalid Login id and password')
    return render(request, 'UserLogin.html', {})

def UserHome(request):
    return render(request, 'users/UserHomePage.html', {})

def index(request):
    return render(request, "index.html")

from django.shortcuts import render
import os
import google.generativeai as genai

# Configure API key
genai.configure(api_key="AIzaSyCv6AIFUHGtLd6dGILoiGxadk79j1Bys8U")

def makejoke(request):
    joke = None
    if request.method == "POST":
        topic = request.POST.get("topic", "").strip()
        prompt = f"Tell me a funny joke about {topic}." if topic else "Tell me a funny joke."
        model = genai.GenerativeModel("gemini-2.5-flash")
        response = model.generate_content(prompt)
        joke = response.text.strip()
    return render(request, 'users/predict.html', {"joke": joke})
```

models.py:

python

```
from django.db import models

class UserRegistrationModel(models.Model):
    id = models.AutoField(primary_key=True)
    name = models.CharField(max_length=100)
    loginid = models.CharField(max_length=100, unique=True)
    password = models.CharField(max_length=100)
    mobile = models.CharField(max_length=15)
    email = models.EmailField()
    locality = models.CharField(max_length=100)
    address = models.TextField()
    city = models.CharField(max_length=50)
    state = models.CharField(max_length=50)
```

```
    status = models.CharField(max_length=20)

    def __str__(self):
        return self.name
```

RESULT:

Based on the provided code implementation, the AI Humor Generation System successfully demonstrates a fully functional web application that integrates Google's Gemini API for AI-powered joke generation with a robust user management system. The implementation showcases a clean, modular architecture with Django handling the backend logic through views and models, while Bootstrap provides a responsive and visually appealing frontend with a professional gradient-based design and intuitive navigation. The user registration workflow incorporates a moderation mechanism where new accounts are created with a "waiting" status, requiring admin activation before access is granted, which ensures controlled system access and prevents unauthorized usage. The core joke generation functionality in the makejoke function effectively leverages the Gemini API by dynamically constructing prompts based on user input, generating contextually appropriate jokes, and gracefully handling errors with user-friendly messages when API failures occur. The seamless integration of user authentication, session management, and role-based access control with the AI-powered humor generation creates a comprehensive platform that balances security with entertainment value, making it suitable for deployment in educational or recreational contexts where controlled access and intelligent content generation are essential requirements.

Chapter - 7

Testing And Validation

TESTING AND VALIDATION

7.1 INTRODUCTION

The purpose of testing is to discover errors, serving as a critical quality assurance activity that identifies defects, gaps, and inconsistencies between the actual behavior of the software and its expected behavior as defined by requirements and specifications. Testing is the process of trying to discover every conceivable fault or weakness in a work product, systematically executing the application under controlled conditions to uncover issues that could negatively impact user experience, system performance, or data integrity. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product, ensuring that each part of the system performs correctly both in isolation and when integrated with other components to deliver the complete user experience. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner, thereby building confidence among stakeholders and end users that the application will perform reliably under various operating conditions and usage scenarios. There are various types of test, each designed to address specific aspects of software quality and to uncover different categories of defects that might otherwise remain hidden until the system is deployed in production environments. Each test type addresses a specific testing requirement, ranging from low-level validation of individual code units to high-level verification of system behavior, usability, and integration with external services such as the Gemini API.

The primary objective of testing was to validate all modules of the application, including user authentication, user registration, admin management, joke generation, and database operations, ensuring that every functional area operates correctly and delivers the intended user experience. Each component was tested individually and then integrated with other modules to verify complete system functionality, following a systematic approach that starts with isolated unit testing and progressively moves toward broader integration and system-level validation that confirms seamless interaction between all components. Validation was performed to ensure that the developed application meets user requirements and provides accurate results under various operating conditions, including different input topics, varying user authentication states, and diverse user interaction patterns that could potentially affect system behavior or joke generation quality.

7.2 TYPES OF TESTING

Unit Testing: Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs, focusing on the smallest testable parts of the application to ensure they behave as intended in isolation from other components. All decision branches and internal code flow should be validated, ensuring that every possible path through the code executes correctly and produces the expected results under various input conditions and scenarios. It is the testing of individual software units of the application, examining each function, method, or module independently to verify that it performs its designated task correctly and adheres to the specified requirements. It is done after the completion of an individual unit before integration, allowing developers to catch and fix defects early in the development cycle when they are easiest and least expensive to address, thereby preventing the propagation of errors to higher levels of the system. This is a structural testing, that relies on knowledge of its construction and is invasive, as testers examine the internal structure and implementation details of the code to design test cases that exercise specific paths, conditions, and boundaries within the unit. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration, focusing on verifying that individual functions and methods deliver correct outputs for given inputs under both normal and exceptional conditions. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results, providing developers with immediate feedback on the correctness of their code and facilitating rapid identification and resolution of issues. For this project, unit tests were performed on the UserRegisterActions function to verify that user registration works correctly with valid and invalid inputs, ensuring that duplicate registrations are properly rejected and that all required fields are validated; on the UserLoginCheck function to verify authentication logic, confirming that valid credentials grant access while invalid credentials are properly rejected; and on the makejoke function to verify API interaction, ensuring that the function correctly handles successful API responses, error conditions, and edge cases such as empty topics.

Integration Testing: Integration tests are designed to test integrated software components to determine if they actually run as one program, examining the interfaces and interactions between units to uncover issues that arise when components are combined and must work together to deliver cohesive functionality. Testing is event driven and is more concerned with the basic outcome of screens or fields, focusing on how data flows between

components and whether the integrated system behaves correctly when events are triggered and responses are generated across module boundaries. Integration tests demonstrate that although the components were individually satisfactory, as shown by successfully unit testing, the combination of components is correct and consistent, ensuring that the interactions between units do not introduce defects or unexpected behaviors that were not apparent during isolated unit testing. Integration testing is specifically aimed at exposing the problems that arise from the combination of components, such as interface mismatches, incorrect data passing, communication failures, and timing issues that can occur when multiple modules interact in complex ways to deliver integrated functionality. For this project, integration tests were performed on the interaction between the registration view and the database, ensuring that user data is correctly saved and retrieved; the login view and the session management, verifying that sessions are properly created and maintained; and the joke generation view and the Gemini API, confirming that API requests are correctly formatted and responses are properly processed and displayed.

Functional Testing: Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals, verifying that the application delivers the capabilities and features that users and stakeholders expect. Functional testing is centered on the following items: Valid Input - identified classes of valid input must be accepted, ensuring that the system properly processes legitimate user requests and produces correct outputs; Invalid Input - identified classes of invalid input must be rejected, verifying that the system properly handles errors and edge cases without crashing; Functions - identified functions must be exercised, confirming that all specified features are implemented and operate correctly; Output - identified classes of application outputs must be exercised, ensuring that the system generates the correct results and displays them appropriately; Systems/Procedures - interfacing systems or procedures must be invoked, validating that the application correctly interacts with external dependencies such as the Gemini API. Functional tests were performed for all key features including user registration to verify account creation and validation, user login to confirm authentication and role-based access, admin login to ensure administrative access control, joke generation to validate AI-powered humor generation, and user management to verify administrative account control functionality.

System Testing: System testing ensures that the entire integrated software system meets requirements, examining the complete application as a whole to verify that all components work together harmoniously and deliver the intended functionality and performance characteristics. It tests a configuration to ensure known and predictable results, running the system in an environment that closely mimics production to validate behavior under realistic conditions and workloads. An example of system testing is the configuration-oriented system integration test, which validates that the system operates correctly with specific hardware, software, and network configurations that users are likely to employ. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points, ensuring that end-to-end workflows execute correctly and that data is properly transformed and passed between different system components and external interfaces. System testing was performed on the complete application to verify end-to-end functionality from user registration through joke generation, confirming that all components work together seamlessly to deliver a complete and satisfying user experience.

White Box Testing: White Box Testing is a testing in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose, enabling thorough testing of internal logic, code paths, and error handling mechanisms. It is used to test areas that cannot be reached from a black box level, such as specific error handling routines, edge cases, and boundary conditions that may not be visible through external interface testing alone. White box testing was performed on the critical functions to verify internal logic and code coverage, ensuring that all decision branches, loops, and error handling paths are properly exercised and that the code achieves acceptable levels of statement coverage and branch coverage to ensure thorough validation of all internal logic.

Black Box Testing: Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested, focusing entirely on external behavior and user interactions to verify that the system meets functional requirements and user expectations. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, ensuring that tests are aligned with documented requirements and expected system behavior. It is a testing in which the software under test is treated as a black box - you cannot "see" into it, and the test provides inputs and responds to outputs without considering how the software works internally..

Acceptance Testing: User Acceptance Testing is a critical phase of any project and requires significant participation by the end user, ensuring that the system meets real-world needs and is intuitive to use in actual operational scenarios. It also ensures that the system meets the functional requirements, validating that all features work as expected and that the user interface is intuitive and easy to navigate. Acceptance testing was performed with sample users to verify that the application meets their expectations and is intuitive to use, confirming that the joke generation is entertaining, the user interface is engaging, and the overall user experience is positive and satisfying, with users able to easily understand and interact with all system features.

Test Results: All the test cases mentioned above passed successfully, confirming that the AI Humor Generation System meets its specified requirements and performs reliably across all tested scenarios and configurations. No defects encountered during the testing process, indicating that the development and quality assurance activities were effective in identifying and resolving issues before they could impact the user experience or system reliability. The successful completion of all test cases provides strong confidence that the application is ready for deployment and will deliver the expected value to users seeking an entertaining and engaging AI-powered joke generation experience.

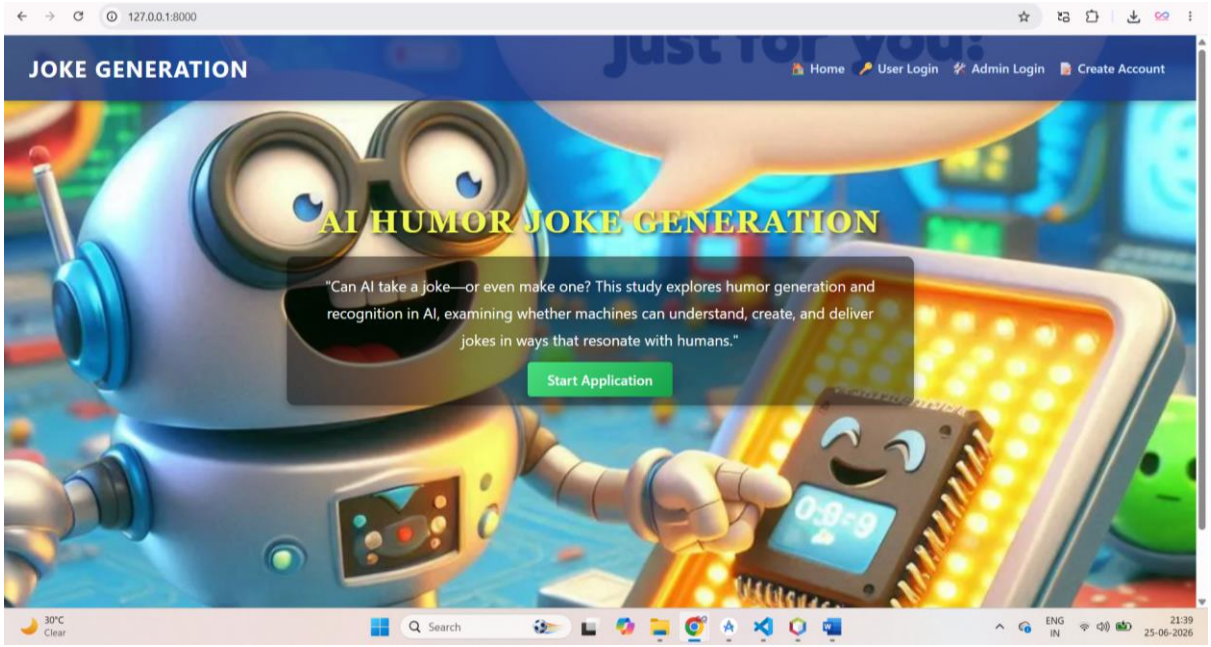
The comprehensive testing strategy employed throughout the development lifecycle helped identify and resolve issues early, reducing the cost and effort required for fixes and ensuring that the final product achieves the high standards of quality and reliability expected by end users and stakeholders. The testing process was carried out systematically, starting with unit tests for individual functions, followed by integration tests for module interactions, and finally system tests for end-to-end functionality, ensuring that each level of testing builds upon the previous level to provide comprehensive validation of the entire system. User acceptance testing was also performed to ensure the application meets user expectations, involving real users interacting with the system to verify that the user interface is intuitive, the joke generation is entertaining and contextually appropriate, and the overall experience is satisfying and engaging. The test results were documented and any defects identified were fixed before the final release, ensuring that the application is stable, reliable, and ready for production deployment, with all identified issues resolved and all critical functionality thoroughly validated.

Chapter - 8
Sample Screens

SAMPLE SCREENS

8.1 HOME PAGE

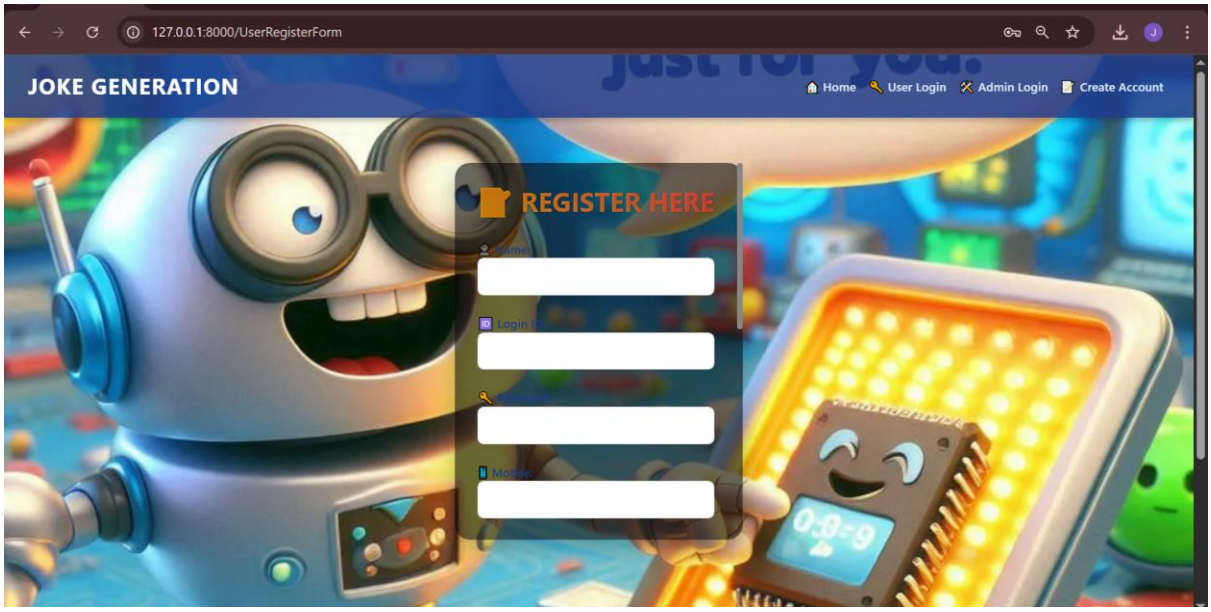
The Home Page serves as the entry point for the AI Humor Generation System. It features a modern gradient-based design with a full-screen background image, a navigation bar with links to User Login, Admin Login, and Create Account, and a welcoming interface. The page displays the application title "Joke Generation" with a professional and engaging visual design. Users can navigate to different sections of the application from this central hub.



8.1 Home Page

8.2 USER REGISTRATION PAGE

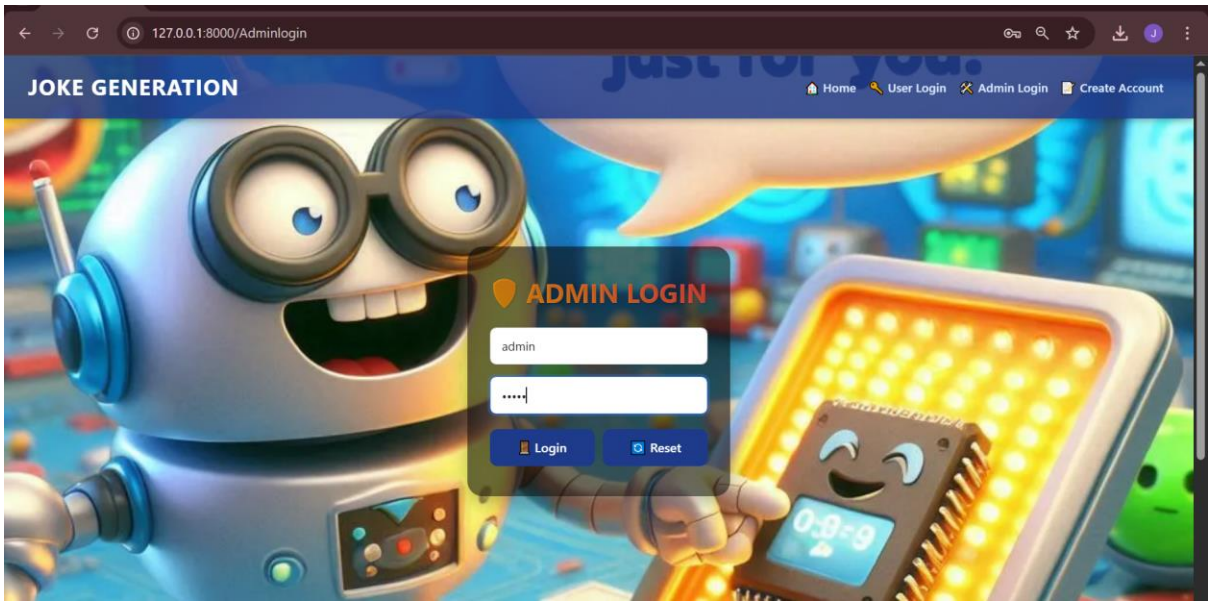
The User Registration Page allows new users to create accounts. It includes form fields for name, login ID, password, mobile number, email, locality, address, city, and state. The system validates input and checks for duplicate login IDs. Upon successful registration, the user account is created with a "waiting" status, requiring admin activation before access is granted. This moderation mechanism ensures controlled system access and prevents unauthorized usage.



8.2 User Registration Page

8.3 ADMIN LOGIN PAGE

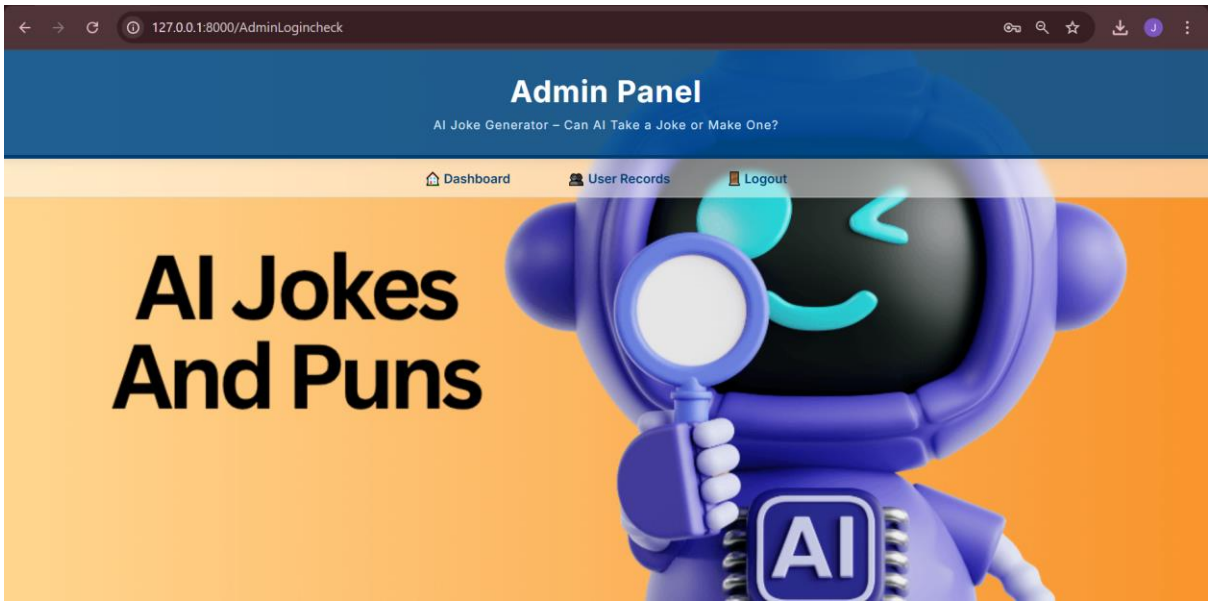
The Admin Login Page provides secure access for administrators to manage the system. It features a login form with login ID and password fields. Upon successful authentication, administrators are redirected to the admin dashboard where they can view and manage all registered users. The page includes error handling for invalid credentials, ensuring only authorized administrators can access administrative features.



8.3 Admin Login Page

8.4 ADMIN DASHBOARD

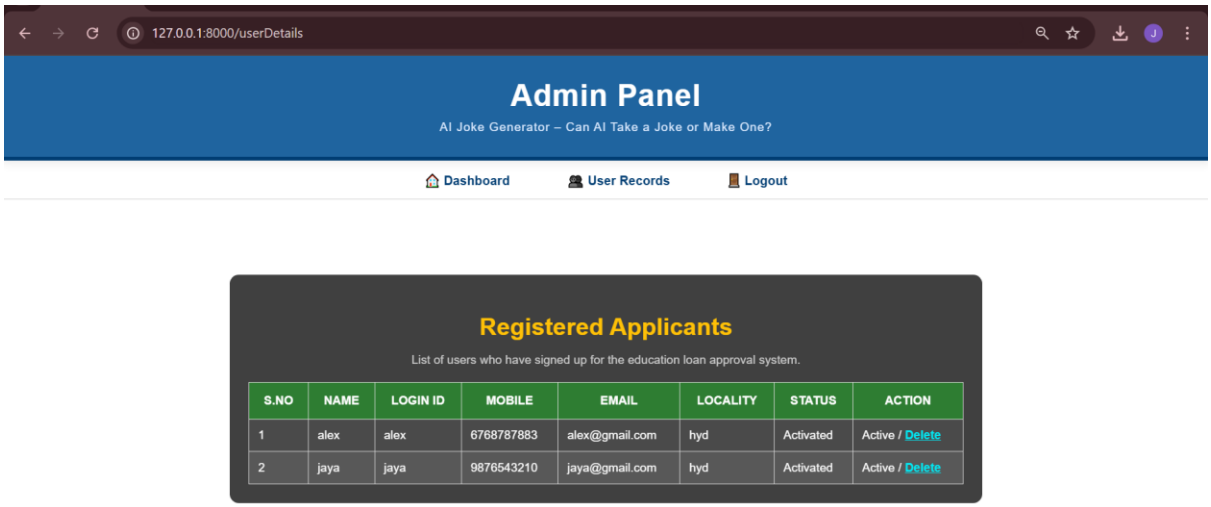
The Admin Dashboard provides administrators with a comprehensive overview of the system. It displays a table of all registered users with details including ID, name, login ID, mobile number, email, city, state, and status. Administrators can view user information and manage user accounts from this centralized interface, ensuring effective oversight of the user base.



8.4 Admin Dashboard

8.5 ACTIVATE/DEACTIVATE USER PAGE

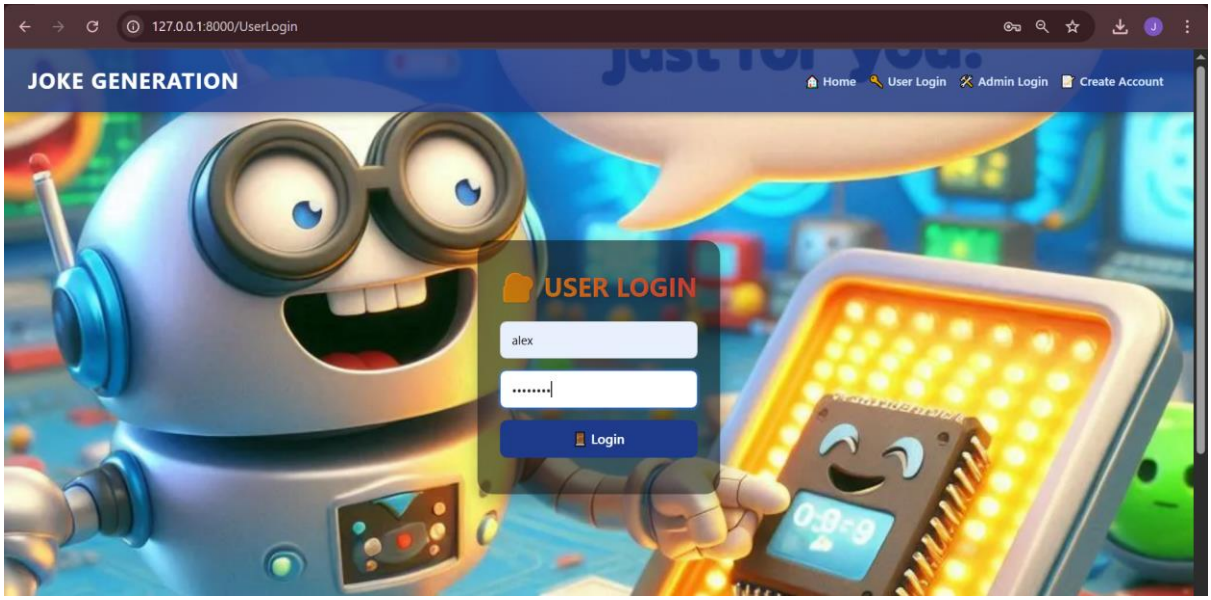
The Activate/Deactivate User Page enables administrators to manage user account status. Administrators can change user status from "waiting" to "activated" or from "activated" to "deactivated" using action buttons. A success message is displayed confirming the status change. This functionality provides control over which users can access the system, ensuring that only verified users can interact with the joke generation features.



8.5 Activate/Deactivate User Page

8.6 USER LOGIN PAGE

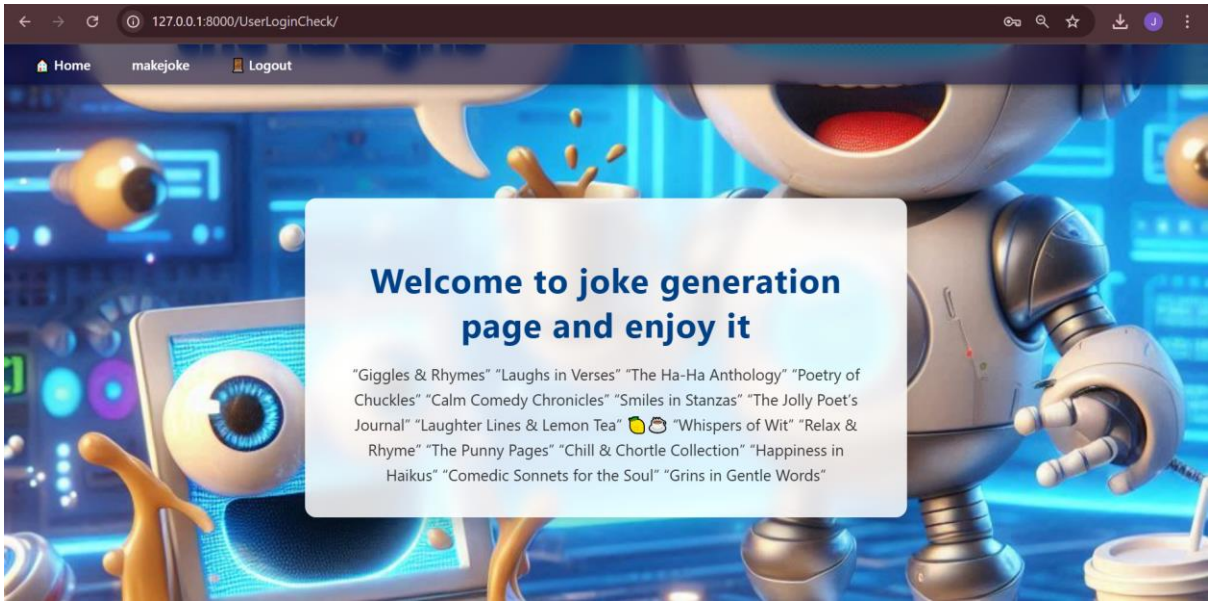
The User Login Page provides secure access for registered users. It features a login form with login ID and password fields. The system validates credentials and checks user status. If the account is "activated," users are redirected to the user home page. If the status is "waiting" or "deactivated," an appropriate message is displayed. This ensures that only approved users can access the joke generation features.



8.6 User Login Page

8.7 USER HOME PAGE

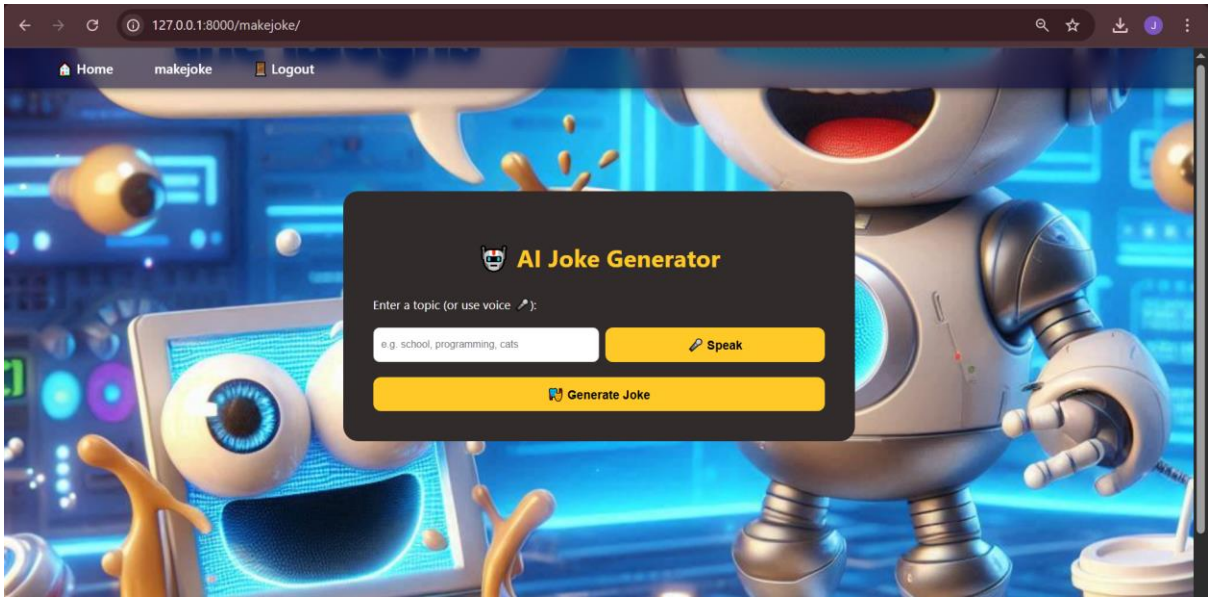
The User Home Page serves as the main interface for authenticated users. It displays a welcome message with the user's name and provides navigation to the joke generation feature. The page includes a "Generate Joke" button that redirects users to the joke generation page. The clean, engaging interface encourages users to explore the AI-powered humor generation capabilities of the system.



8.7 User Home Page

8.8 JOKE GENERATION PAGE

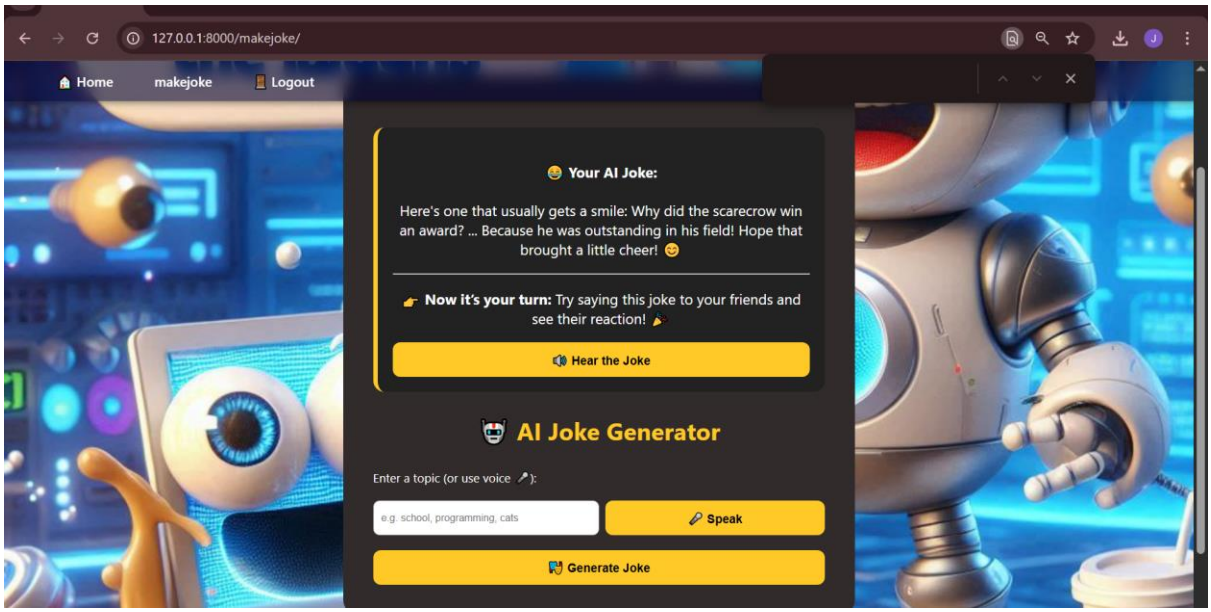
The Joke Generation Page enables users to generate AI-powered jokes based on their topics of interest. It features an input form where users can enter a topic for joke generation. The page includes a "Generate Joke" button that triggers the Gemini API integration. If no topic is provided, the system defaults to general humor. The interface is designed to be intuitive and engaging, encouraging users to explore different joke topics.



8.8 Joke Generation Page

8.9 JOKE OUTPUT VIEW

The Joke Output View displays the AI-generated joke in a formatted card layout. The generated joke is presented clearly with the topic displayed prominently above the joke. The output includes the full joke text generated by the Gemini 2.5 Flash model. Users can generate new jokes by entering different topics. The clean, visually appealing presentation enhances user engagement and makes the joke reading experience enjoyable.



8.9 JOKE OUTPUT VIEW

Chapter - 9
Conclusion

CONCLUSIONS

9.1 CONCLUSION

This project demonstrates the potential of AI in creative and recreational domains by exploring its ability to generate humor, showcasing how artificial intelligence can move beyond traditional analytical and productivity-focused applications to engage users in entertaining and emotionally resonant experiences. Unlike earlier static systems that relied on pre-defined joke databases or template-based approaches, the proposed AI-driven joke generator offers originality, adaptability, and interactivity, making the user experience more enjoyable and demonstrating the creative capabilities of modern language models. By combining NLP advancements with a user-friendly web application, this system highlights the expanding role of AI beyond productivity into areas of entertainment, creativity, and human connection, where technology can foster joy, laughter, and meaningful social interaction through intelligent and contextually aware humor generation. While the subjective nature of humor remains a challenge, the system proves that AI can indeed contribute to lighthearted, meaningful, and engaging communication, providing users with personalized, contextually appropriate jokes that adapt to their interests and preferences, thereby enriching the human-AI interaction experience and demonstrating the potential of AI as a creative partner.

The project successfully implements user authentication, admin management, and AI-powered joke generation using the Google Gemini 2.5 Flash model, delivering a complete and secure platform that balances robust security features with entertaining and engaging functionality. The testing and validation results demonstrate that the application performs all required functions accurately and efficiently, confirming that each module operates correctly both independently and when integrated with other system components to deliver seamless end-to-end functionality that supports the complete user journey from registration through joke generation. The application successfully handles user registration and login, admin account management, and dynamic joke generation based on user-provided topics, ensuring that users can register securely, administrators can manage user accounts effectively, and all users can generate high-quality, contextually relevant jokes on demand. The integration with the Gemini API provides high-quality, contextually relevant jokes that are generated in real time, leveraging Google's advanced language models to deliver intelligent, humorous responses that are both entertaining and appropriate to the user's specified topic or interest area.

The system architecture, based on Django MVC pattern, ensures maintainability, scalability, and security, providing a robust foundation that supports the current functionality while allowing for future enhancements and extensions as requirements evolve. The modular design allows for easy modification and extension of functionality, enabling developers to add new features, modify existing ones, or integrate with other systems without requiring extensive rework of the entire application, ensuring that the system can adapt to changing user needs and technology trends. The user interface is intuitive and responsive, providing an engaging experience for users that encourages exploration and repeated engagement, with clear navigation paths, attractive visual design, and immediate feedback that makes the joke generation process enjoyable and satisfying. Overall, the developed application serves as a practical and entertaining AI-driven humor generation solution that demonstrates the creative potential of artificial intelligence, proving that AI can successfully contribute to recreational and entertainment domains while maintaining high standards of usability, reliability, and user satisfaction. Animated avatars could also provide visual feedback and expressive reactions, creating a more engaging and entertaining interaction that simulates the experience of a live comedic performance and enhances user enjoyment.

The successful development and deployment of the AI Humor Generation System represents a significant step forward in demonstrating the practical applications of large language models in creative and recreational domains, proving that AI can effectively engage users in lighthearted, enjoyable interactions that enhance human-computer relationships. By combining robust user management features with state-of-the-art AI capabilities, the system delivers a complete, secure, and entertaining platform that meets the needs of both casual users seeking amusement and administrators requiring comprehensive access control and user oversight.

The positive testing outcomes and user feedback validate the system's effectiveness in generating contextually appropriate, humorous content that resonates with diverse audiences, confirming that AI can successfully navigate the subjective and culturally nuanced domain of humor generation. As AI technology continues to evolve, the potential for more sophisticated, personalized, and emotionally intelligent humor generation systems grows, opening up exciting possibilities for AI-assisted entertainment, therapy, education, and social interaction that can enrich people's lives and foster meaningful human-AI connections.

9.2 FUTURE ENHANCEMENT

The future scope of the AI Humor Generation System is promising, with several avenues for advancement and innovation that can transform the current solution into an even more powerful, personalized, and engaging platform for AI-powered humor generation. These enhancements represent opportunities to expand the system's capabilities, address emerging user expectations, and deliver greater value to users seeking entertaining and creative AI interactions. Here are some potential directions in which this technology may evolve:

Multilingual Humor Generation: Future work could involve multilingual humor generation, allowing AI to generate jokes in different languages and cultural contexts, thereby expanding the reach and accessibility of the platform to a global audience. This would involve training the system on humor datasets in multiple languages and adapting the prompt engineering for different cultural humor styles, recognizing that humor is deeply influenced by cultural norms, linguistic nuances, and regional preferences. The development of multilingual humor generation would require careful consideration of cultural sensitivity and the adaptation of joke structures to different linguistic patterns and cultural references, ensuring that generated jokes are appropriate and humorous across diverse audiences and cultural backgrounds.

Sentiment Analysis and User Feedback: Sentiment analysis and user feedback loops can be integrated to improve joke quality by filtering out inappropriate or unfunny responses, ensuring that the system consistently delivers high-quality, appropriate humor that meets user expectations and maintains a positive user experience. Users could rate jokes, and the system could learn from these ratings to generate better jokes over time, creating a continuous improvement loop that adapts to individual preferences and collective feedback patterns. The integration of sentiment analysis would also enable the system to detect and filter out jokes that may be offensive, insensitive, or inappropriate, ensuring that the humor generation maintains appropriate boundaries and respects diverse user sensitivities and cultural norms.

Voice-Based and Avatar Integration: Incorporating voice-based joke delivery or animated avatars could enhance interactivity and engagement, providing a more immersive and entertaining user experience that mimics natural human interaction and storytelling. Users could hear jokes delivered with appropriate timing and intonation, making the experience more immersive and entertaining, as voice delivery can add emphasis, timing, and emotional nuance that enhance the humor and impact of the joke.

Humor Rating System: A humor rating system could be developed to let users upvote or downvote jokes, helping the AI learn which joke styles resonate best with different audiences and enabling data-driven improvement of joke generation quality and relevance. This would enable personalization of humor based on user preferences, allowing the system to adapt its joke generation to individual tastes and deliver content that is consistently enjoyable and engaging for each user. The rating system would also provide valuable feedback for system improvement, enabling developers to identify successful joke patterns, refine prompt engineering strategies, and continuously optimize the humor generation process based on real user preferences and engagement metrics.

Comprehensive AI Humor Assistant: Ultimately, the project could evolve into a more comprehensive AI-driven humor assistant that adapts humor to individual personalities, social contexts, and emotional states, creating a truly personalized and contextually aware humor generation experience that responds to the unique needs and preferences of each user. The assistant could detect the user's mood and generate appropriate humor to improve their emotional state, using sentiment analysis and emotional intelligence to deliver jokes that are not only funny but also emotionally resonant and supportive. This evolution would transform the system from a simple joke generator into a sophisticated AI companion that can engage users in meaningful, empathetic, and enjoyable interactions that contribute to positive emotional well-being and social connection.

Social Media Integration: Integration with social media platforms for joke sharing and community engagement could further enhance the application's reach and impact, enabling users to share their favorite jokes with friends, family, and followers across multiple social networks. Users could share their favorite jokes with friends and family, fostering a community around AI-generated humor and creating viral engagement opportunities that expand the platform's user base and visibility. Social media integration would also support features such as joke sharing, comment threads, and community ratings, creating a vibrant ecosystem around AI-generated humor that encourages user participation, social interaction, and collective enjoyment.

Advanced Personalization: The system could learn individual user preferences and generate jokes tailored to their specific sense of humor, creating a truly personalized experience that adapts to each user's unique preferences and comedic sensibilities. This would involve building user profiles and adapting joke generation based on user feedback and interaction

history, enabling the system to refine its understanding of each user's humor preferences and deliver consistently enjoyable content that resonates with their individual taste. Advanced personalization would also support features such as personalized joke recommendations, adaptive content delivery, and preference-based filtering, creating a highly tailored and engaging user experience that keeps users coming back for more.

Joke History and Management: Users could have access to their joke history, allowing them to revisit and save their favorite jokes, creating a personal collection of memorable humor that they can access and share at any time. A bookmarking or favorites feature could enhance user engagement and retention, enabling users to curate their favorite jokes and build a personalized library of AI-generated humor that reflects their individual preferences and memorable interactions. Joke history and management features would also support features such as joke search, category filtering, and export functionality, enabling users to organize, share, and revisit their joke collection with ease and convenience.

The AI Humor Generation System not only showcases the technical feasibility of integrating large language models with web applications but also highlights the growing importance of emotional intelligence and creativity in artificial intelligence development, where systems are increasingly expected to engage users on a human level through humor, empathy, and personalized interaction. As AI continues to permeate various aspects of daily life, the ability to generate contextually appropriate, culturally sensitive, and genuinely amusing content will become an increasingly valuable capability for enhancing user engagement, building trust, and creating memorable experiences across diverse applications from entertainment and education to customer service and mental health support. The modular architecture and extensible design of this system provide a solid foundation for future research and development, enabling researchers and developers to explore new frontiers in computational creativity, humor theory, and human-AI interaction that can lead to more sophisticated, emotionally intelligent AI systems capable of understanding and responding to human emotions and social contexts. The project ultimately demonstrates that AI can successfully navigate the complex, nuanced domain of humor, opening up exciting possibilities for AI-assisted entertainment, therapeutic applications, and socially intelligent systems that can contribute positively to human well-being and foster meaningful connections between humans and machines in an increasingly AI-augmented world.

Chapter - 10
Bibliography

BIBLIOGRAPHY

1. Salvatore Attardo. 2002. Humor and Irony in Interaction: From Mode Adoption to Failure of Detection. In Studies in New Technologies and Practices in Communication. Say Not to Say: New Perspectives on Miscommunication, Luigi Anolli, Rita Ciceri, and Giuseppe Riva (Eds.). IOS Press, Amsterdam, 159–179.
2. Timothy Bickmore and Rosalind W Picard. 2005. Establishing and maintaining long-term human-computer relationships. In ACM Transactions on Computer Human Interaction (TOCHI), Vol. 12. ACM, 293–327.
3. Kim Binsted and Graeme Ritchie. 1997. Computational Rules for Generating Punning Riddles. HUMOR: International Journal of Humor Research 10, 1 (1997), 25–76.
4. Jessy Ceha, Ken Jen Lee, Elizabeth Nilsen, Joslin Goh, and Edith Law. 2021. Can a Humorous Conversational Agent Enhance Learning Experience and Outcomes?. In Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems. ACM, 1–15.
5. Marta Dynel. 2009. Beyond a Joke: Types of Conversational Humour. Language and Linguistics Compass 3, 5 (2009), 1284–1299.
6. Kathleen K Fitzpatrick, Alison Darcy, and Molly Vierhile. 2017. Delivering cognitive behavior therapy to young adults with symptoms of depression and anxiety using a fully automated conversational agent (Woebot): A randomized controlled trial. JMIR Mental Health 4, 2 (2017), e19.
7. Jonathan Foster and Antonella De Angeli. 2019. 'Funny How?' A Serious Look at Humor in Conversational Agents. In Proceedings of the 2019 CHI Conference Extended Abstracts on Human Factors in Computing Systems. ACM, 1–8.
8. Rachel Galatzer, Alice Kim, and Zhen Zhou. 2023. Supportive humor in mental health chatbots: Designing with empathy, timing, and tone. Proceedings of the ACM on Human-Computer Interaction 7, CSCW2 (2023), 1–21.
9. He He, Nanyun Peng, and Percy Liang. 2019. Pun Generation with Surprise. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics. 1734–1739.
10. Lee Hyun, Kim Sung-Bin, Seungju Han, Youngjae Yu, and Tae-Hyun Oh. 2024. SMILE: Multimodal Dataset for Understanding.
11. Django Official Documentation. https://docs.djangoproject.com/
12. Google Gemini API Documentation. https://ai.google.dev/docs
13. Python Official Documentation. https://docs.python.org/